PROYECTOS JAVA · INTEGRACIÓN GRÁFICA

Juego del Totito

Interfaz neón en Java Swing integrada al código original
Explicación archivo por archivo y bloque por bloque



Repositorio: **AnshonyC/proyectos-java** · carpeta **Totito/**

Fecha: 12 de octubre de 2026 · Java 17 o superior · Swing + Java2D, sin dependencias

Contenido

1. Qué se hizo y por qué

2. Diseño frente a resultado

3. Cómo ejecutarlo

4. Mapa de archivos

5. Arquitectura: cómo encajan las piezas

6. Cómo funciona por dentro (recorridos)

7. De JuegoMain a la versión nueva

8. Cómo leer la explicación del código

Parte A — El código original (base del juego)

Tablero.java · Jugador.java · IA_Minimax.java

Parte B — Lógica nueva: Partida y Navegacion

Partida.java · Navegacion.java

Parte C — Arranque y cambio de pantallas

TotitoApp.java · PanelPrincipal.java

Parte D — El estilo: colores, dibujo e iconos

Tema.java · Dibujo.java · Iconos.java

Parte E — Componentes reutilizables

Tarjeta.java · CampoTexto.java · OpcionModo.java
BotonNeon.java · PanelTablero.java · PanelMarcador.java
Encabezado.java · TarjetaJugador.java · PasoInstruccion.java
Rotulo.java · SelectorSimbolo.java · IconoResultado.java

Parte F — Las pantallas

PantallaBase.java · PantallaInstrucciones.java · PantallaJuego.java
PantallaMenu.java · PantallaConfiguracion.java · PantallaResultado.java

Parte G — Configuración del proyecto

pom.xml

Glosario de Java y Swing

Pruebas realizadas

Cómo seguir trabajando

1. Qué se hizo y por qué

Se partió de dos cosas: el **código del Totito** (un juego de Jugador contra IA con una ventana Swing sencilla) y **cuatro imágenes de diseño** con una interfaz de estilo neón: menú, instrucciones, configuración y pantalla de victoria. El objetivo era que el juego se viera como el diseño **sin rehacer el código existente**.

> **Decisión principal: el código original no se tocó.**
> `Tablero.java`, `Jugador.java`, `IA_Minimax.java` y `JuegoMain.java` están exactamente como llegaron. Se pueden comparar en el historial de Git: el primer commit de la carpeta los sube tal cual y el segundo agrega todo lo nuevo. La interfaz nueva *usa* esas clases desde afuera.

Lo que se agregó

- **2679 líneas nuevas** en 26 archivos: 25 archivos Java (clases e interfaces) y el `pom.xml`.
- **Las 4 pantallas del diseño**, y además una **pantalla de partida** (el tablero mientras se juega), que el diseño no traía y se hizo con el mismo estilo.
- Una clase de lógica, `Partida`, que une las clases originales: turnos, ganador, empate, marcador y los dos modos de juego.
- **Dos modos de juego**: «2 jugadores» (el del diseño) y «Contra la IA», agregado para seguir usando el `IA_Minimax` que ya tenía el proyecto.
- **Todo dibujado con código** (Java2D): fichas, botones, iconos, trofeo y confeti. No hay imágenes ni librerías externas; para ejecutarlo solo se necesita el JDK.
- Documentación: comentarios en cada clase y método, el `README.md` con capturas y este documento.

Decisiones de diseño y su motivo

Decisión	Por qué
Las clases nuevas van en `src/` sin paquetes	El código original no tiene `package`. Java no permite que una clase con paquete use una que no lo tiene, así que, para no modificar el original, todo quedó en el mismo nivel. El `pom.xml` apunta a esa carpeta.
Separar lógica (`Partida`) e interfaz (pantallas)	En `JuegoMain` las reglas y la ventana estaban mezcladas. Separadas, las reglas se prueban sin abrir ventanas, y la interfaz se puede cambiar sin romper el juego.
Dibujar todo con Java2D en lugar de usar imágenes	Se ve nítido a cualquier tamaño, no hay archivos que se puedan perder y los colores se cambian en un solo lugar (`Tema`).
Pantallas que no se conocen entre sí (interfaz `Navegacion`)	Para agregar o cambiar una pantalla no hay que tocar las demás.
Ventana fija de 640×690 píxeles	El diseño es de tamaño fijo, y 690 px de alto caben en pantallas de 1366×768, comunes en laptops.
Siempre empieza la X	Es la regla tradicional. Si el Jugador 1 elige la O, abre el Jugador 2 (o la computadora).

2. Diseño frente a resultado

A la izquierda, la imagen de diseño recibida; a la derecha, la captura del programa en Java. Las capturas se generaron con el mismo código del repositorio.

1. Menú principal



Diseño



Resultado en Java

2. Instrucciones



Diseño



Resultado en Java

3. Configuración de la partida



Diseño



Resultado en Java

4. Pantalla de victoria



Diseño



Resultado en Java

Pantallas que no estaban en el diseño



Partida en curso: marcador, turno y ficha «fantasma» bajo el ratón



Resultado en caso de empate

Diferencias a propósito con el diseño: en «Modo de juego» se agregó **Contra la IA**, y bajo los símbolos se agregó una leyenda («Jugador 1 juega con X…») para que no haya dudas. La letra es la mejor disponible en cada equipo (Segoe UI en Windows), porque Java no trae la fuente redondeada de la imagen.

3. Cómo ejecutarlo

Requisito único: JDK 17 o más nuevo (Oracle, Temurin o el que trae NetBeans). Se comprueba con `java -version`. No hay que instalar ninguna librería.

Forma	Pasos
NetBeans / IntelliJ	*File* → *Open Project* → carpeta `Totito` (se abre como proyecto Maven) → botón **Run**. Arranca `TotitoApp`.
Maven en terminal	`cd Totito` → `mvn package` → `java -jar target/Totito-1.0-SNAPSHOT.jar`
Solo con el JDK	`cd Totito` → `javac -encoding UTF-8 -d out src/*.java` → `java -cp out TotitoApp`
Versión original	Después de compilar: `java -cp out JuegoMain` (la ventana sencilla de antes, que sigue funcionando).

> **Importante:** al compilar a mano hay que poner `-encoding UTF-8`. Los textos llevan tildes y signos (¡, ó, í) y, sin esa opción, en Windows se ven caracteres raros. NetBeans y Maven ya lo hacen solos gracias al `pom.xml`.

Cómo se juega

1. **Jugar** → se escriben los nombres. Si se dejan vacíos, quedan «Jugador 1» y «Jugador 2».
2. **Símbolos**: la caja marcada es la del Jugador 1 y el Jugador 2 usa la otra. Empieza siempre la X.
3. **Modo**: «2 jugadores» (turnos en el mismo equipo) o «Contra la IA» (el Jugador 2 es la computadora).
4. Al formar tres en línea aparece la raya y, un momento después, la pantalla de resultado.
5. **Jugar de nuevo** conserva a los jugadores y suma al marcador. **Menú principal** empieza de cero.

4. Mapa de archivos

Todos los archivos de `Totito/` que se explican en este documento, en el orden en que aparecen. El nombre es un enlace a su explicación.

Archivo	Qué es	Líneas	Estado
`Tablero.java`	Modelo (original)	92	original, sin cambios
`Jugador.java`	Modelo (original)	22	original, sin cambios
`IA_Minimax.java`	Inteligencia artificial (original)	97	original, sin cambios
`Partida.java`	Modelo nuevo (controla la partida)	160	nuevo
`Navegacion.java`	Interfaz (contrato entre pantallas)	26	nuevo
`TotitoApp.java`	Punto de entrada (main)	48	nuevo
`PanelPrincipal.java`	Controlador de pantallas	91	nuevo
`Tema.java`	Paleta de colores y fuentes	103	nuevo
`Dibujo.java`	Utilidades de dibujo (el efecto neón)	166	nuevo
`Iconos.java`	Iconos dibujados con código	268	nuevo
`Tarjeta.java`	Panel "de cristal"	56	nuevo
`BotonNeon.java`	Botón con estilo neón	192	nuevo
`Encabezado.java`	Barra superior	77	nuevo
`Rotulo.java`	Textos decorativos	166	nuevo
`CampoTexto.java`	Caja de texto para nombres	97	nuevo
`PanelTablero.java`	Tablero 3×3 dibujado	197	nuevo
`TarjetaJugador.java`	Tarjeta de nombre de jugador	85	nuevo
`SelectorSimbolo.java`	Elegir X u O	107	nuevo
`OpcionModo.java`	Opción de modo de juego	85	nuevo
`PanelMarcador.java`	Marcador de la partida	80	nuevo
`PasoInstruccion.java`	Paso numerado de las instrucciones	50	nuevo
`IconoResultado.java`	Ilustración del resultado	51	nuevo
`PantallaBase.java`	Base de todas las pantallas	116	nuevo
`PantallaMenu.java`	Pantalla del menú (diseño 1)	34	nuevo
`PantallaInstrucciones.java`	Pantalla "Cómo jugar" (diseño 2)	57	nuevo
`PantallaConfiguracion.java`	Pantalla de configuración (diseño 3)	69	nuevo
`PantallaJuego.java`	Pantalla de la partida	134	nuevo
`PantallaResultado.java`	Pantalla de resultado (diseño 4)	125	nuevo
`pom.xml`	Proyecto Maven	39	nuevo

Otros archivos: `README.md` (guía rápida con capturas), `docs/capturas/*.png` (las 6 capturas), `docs/` (este PDF) y, fuera de la carpeta, una línea en el `.gitignore` del repositorio para no subir `Totito/target/` (lo que genera Maven al compilar).

`JuegoMain.java` también es original y no cambió. No se explica línea por línea porque la versión nueva no lo usa; en la sección 7 se cuenta a dónde fue cada una de sus partes.

5. Arquitectura: cómo encajan las piezas

El programa está en capas. Cada capa solo usa la de abajo, nunca la de arriba: las reglas no saben que existe una ventana, y la ventana no sabe cómo piensa la IA.



> **Regla para el que siga:** si algo es una *regla del juego*, va en `Partida` (o en las clases originales). Si es *cómo se ve*, va en una pantalla o un componente. Si es *un color o una fuente*, va en `Tema`.

6. Cómo funciona por dentro

Recorrido de un clic en el tablero

1. El usuario aprieta el ratón sobre una casilla. Swing llama a `mousePressed` en `PanelTablero`, que convierte los píxeles en *fila* y *columna*.
2. `PanelTablero` avisa a su oyente: `oyente.casillaPulsada(fila, columna)`. El oyente es `PantallaJuego.alPulsarCasilla`.
3. `PantallaJuego` ignora el clic si la IA está pensando; si no, llama a `partida.jugar(fila, columna)`.
4. `Partida.jugar` usa las clases originales: `Tablero.marcarCasilla` (¿la casilla estaba libre?), `Tablero.verificarGanador` y `Tablero.tableroLleno`. Con eso actualiza el estado, el marcador y el turno.
5. `PantallaJuego.actualizar()` pasa el nuevo estado a la pantalla: `PanelTablero.mostrar(tablero)` copia las fichas y busca la línea ganadora, el marcador se refresca y el letrero dice de quién es el turno.
6. Cada componente pide `repaint()` y Swing vuelve a llamar a su `paintComponent`, que dibuja las fichas con `Dibujo.trazoNeon`.

Turno de la computadora

1. Después de cada jugada, `PantallaJuego` pregunta `partida.esTurnoDeIA()`.
2. Si es así, bloquea el tablero y programa un `javax.swing.Timer` de 450 ms. No se usa `Thread.sleep` porque congelaría la ventana.
3. Cuando vence el Timer: `partida.jugarTurnoIA()` → `IA_Minimax.calcularMejorJugada(tablero)` (el algoritmo original) → `partida.jugar(...)` con esa casilla. Es el mismo camino que la jugada humana.
4. Si la partida terminó, otro Timer de 1.2 s deja ver la raya y luego `navegacion.mostrarResultado(partida)` cambia de pantalla.

Cambio de pantallas

Al construirse, cada pantalla recibe un objeto `Navegacion`, que en realidad es el `PanelPrincipal`. Un botón como «Jugar» hace `navegacion.irAConfiguracion()`, y `PanelPrincipal` ejecuta `cartas.show(this, "configuracion")`. Las cinco pantallas se crean **una sola vez** al arrancar y solo se van mostrando u ocultando; así se conserva, por ejemplo, lo que se escribió en los nombres.

El efecto neón

`Dibujo.trazoNeon` traza la misma figura 6 veces: primero muy gruesa y casi transparente, y cada vez más fina y opaca. Encima va la línea nítida y, si el trazo es grueso, un centro casi blanco. Al superponerse, las capas transparentes forman el halo. Toda la estética (bordes, fichas, iconos, raya ganadora) sale de esa única función.

7. De JuegoMain a la versión nueva

`JuegoMain.java` era a la vez la ventana, el controlador y parte de las reglas. Se dejó intacto (sigue funcionando), y en la versión nueva cada una de sus responsabilidades pasó a una clase propia:

En JuegoMain (original)	En la versión nueva
`extends JFrame`, la ventana	`TotitoApp` (ventana) + `PanelPrincipal` (contenido y navegación)
`JButton[3][3] botones`	`PanelTablero`: un solo componente que dibuja las 9 casillas y detecta en cuál se hizo clic
`turnoJugador`, `juegoTerminado`	`Partida`: `turno`, `estado` (EN_CURSO / VICTORIA / EMPATE), `ganador`
`manejarClicCasilla(...)`	`PantallaJuego.alPulsarCasilla(...)` → `Partida.jugar(...)`
`Timer` de 350 ms + `turnoIA()`	`PantallaJuego.programarTurnoIA()` (450 ms) → `Partida.jugarTurnoIA()`, que usa el mismo `IA_Minimax`
`verificarFinDePartida()`	`Partida.jugar()` (calcula el estado) + `PantallaJuego.despuesDeJugada()` (decide qué mostrar)
`etiquetaEstado` («Tu turno (X)»)	El `Rotulo` de turno + `PanelMarcador` (nombres, victorias, empates)
`reiniciarPartida()`	Botón «Reiniciar» / «Jugar de nuevo» → `Partida.reiniciar()` (conserva el marcador)
`COLOR_X`, `COLOR_O`, `COLOR_FONDO`…	`Tema`: toda la paleta neón en un solo lugar
Solo Jugador contra IA, la IA siempre con O	Dos modos; el jugador elige su símbolo y la IA toma el otro

8. Cómo leer la explicación del código

Las partes A a G muestran **cada archivo completo**, línea por línea y con su número, partido en bloques. Debajo de cada bloque hay un recuadro que explica qué hacen esas líneas, cómo y por qué.

```
12  public boolean esTurnoDeIA() {
13      return ia != null && turno == 1 && estado == Estado.EN_CURSO;
14  }
```

Líneas 12-14 Así se ve una explicación: la etiqueta morada dice qué líneas del archivo real se están explicando, para buscarlas en NetBeans.

Colores del código: **palabras reservadas**, clases y tipos, textos, números y *comentarios*. Cada archivo lleva una etiqueta nuevo o original, sin cambios.

Orden recomendado para entender el proyecto: primero **Partida** (las reglas), luego **PanelPrincipal** y **PantallaJuego** (cómo se conecta todo) y, al final, los componentes de dibujo.

Parte A — El código original (base del juego)

Estas tres clases venían en el zip y **no se modificaron ni una línea**. Se explican porque toda la interfaz nueva se apoya en ellas: la clase nueva `Partida` las usa tal como están.

Tablero.java original, sin cambios

Modelo (original) · 92 líneas · `Totito/src/Tablero.java`

Guarda las 9 casillas y conoce las reglas básicas: marcar, saber si hay ganador, si el tablero está lleno y hacer copias para que la IA simule jugadas.

```
1  /**
2   * Representa el tablero de 3x3 del juego Totito.
3   * Se encarga unicamente del estado del juego (casillas, validaciones,
4   * deteccion de victoria/empate). No conoce nada de la interfaz grafica
5   * ni de la logica de la IA.
6   */
7  public class Tablero {
8
```

Líneas 1-8 Declaración de la clase. El comentario de arriba deja claro su límite: no sabe nada de la interfaz ni de la IA. Por eso se pudo reutilizar sin cambios.

```
9      public static final char VACIA = ' ';
10
```

Líneas 9-10 `VACIA` es un espacio (`' '`) y marca una casilla libre. Como es `public static final`, las clases nuevas la usan para comparar (`Tablero.VACIA`) en lugar de escribir `' '` a mano.

```
11     private char[][] casillas;
12
```

Líneas 11-12 La matriz 3×3 de caracteres: cada posición tiene `'X'`, `'O'` o `VACIA`. Es `private`: desde fuera solo se toca con los métodos de abajo.

```
13     public Tablero() {
14         casillas = new char[3][3];
15         reiniciar();
16     }
17
```

Líneas 13-17 El constructor crea la matriz y la deja vacía llamando a `reiniciar()`.

```
18     /** Deja el tablero vacio, listo para una nueva partida. */
19     public void reiniciar() {
20         for (int fila = 0; fila < 3; fila++) {
21             for (int columna = 0; columna < 3; columna++) {
22                 casillas[fila][columna] = VACIA;
23             }
24         }
25     }
26
```

Líneas 18-26 Recorre filas y columnas con dos `for` anidados y pone `VACIA` en cada casilla. `Partida.reiniciar()` lo llama al empezar cada partida nueva.

```java
    /**
     * Intenta colocar una ficha en la posicion indicada.
     * Retorna false si la posicion esta fuera de rango o ya esta ocupada,
     * evitando asi tiros duplicados o invalidos.
     */
    public boolean marcarCasilla(int fila, int columna, char ficha) {
        if (fila < 0 || fila > 2 || columna < 0 || columna > 2) {
            return false;
        }
        if (casillas[fila][columna] != VACIA) {
            return false;
        }
        casillas[fila][columna] = ficha;
        return true;
    }

```

Líneas 27-42 Intenta poner una ficha. Devuelve `false` si la posición está fuera de rango o ya está ocupada; así se evitan jugadas inválidas. `Partida.jugar()` se apoya en este `true/false` para saber si el clic contó.

```java
    public char obtenerCasilla(int fila, int columna) {
        return casillas[fila][columna];
    }

```

Líneas 43-46 Devuelve lo que hay en una casilla. Lo usan `PanelTablero` para dibujar y `Partida.buscarLineaGanadora()` para encontrar la raya.

```java
    /** Retorna true si ya no quedan casillas vacias. */
    public boolean tableroLleno() {
        for (int fila = 0; fila < 3; fila++) {
            for (int columna = 0; columna < 3; columna++) {
                if (casillas[fila][columna] == VACIA) {
                    return false;
                }
            }
        }
        return true;
    }

```

Líneas 47-58 Si no queda ninguna casilla vacía, el tablero está lleno. Si además nadie ganó, es empate.

```
59      /**
60       * Revisa las 3 filas, 3 columnas y 2 diagonales.
61       * Retorna la ficha ganadora ('X' u 'O') o VACIA si nadie ha ganado aun.
62       */
63      public char verificarGanador() {
64          for (int i = 0; i < 3; i++) {
65              if (casillas[i][0] != VACIA && casillas[i][0] == casillas[i][1] && casillas[i][1] == casillas[i][2])
    {
66                  return casillas[i][0];
67              }
68              if (casillas[0][i] != VACIA && casillas[0][i] == casillas[1][i] && casillas[1][i] == casillas[2][i])
    {
69                  return casillas[0][i];
70              }
71          }
72          if (casillas[0][0] != VACIA && casillas[0][0] == casillas[1][1] && casillas[1][1] == casillas[2][2]) {
73              return casillas[0][0];
74          }
75          if (casillas[0][2] != VACIA && casillas[0][2] == casillas[1][1] && casillas[1][1] == casillas[2][0]) {
76              return casillas[0][2];
77          }
78          return VACIA;
79      }
80
```

Líneas 59-80 Revisa las 3 filas, las 3 columnas y las 2 diagonales. Devuelve la ficha ganadora o `VACIA` si nadie ganó. Solo dice *quién* ganó, no *dónde*; por eso se agregó `Partida.buscarLineaGanadora()` sin tocar esta clase.

```
81      /** Crea una copia independiente del tablero, usada por el algoritmo Minimax
82       *  para simular jugadas sin afectar la partida real. */
83      public Tablero obtenerCopia() {
84          Tablero copia = new Tablero();
85          for (int fila = 0; fila < 3; fila++) {
86              for (int columna = 0; columna < 3; columna++) {
87                  copia.casillas[fila][columna] = this.casillas[fila][columna];
88              }
89          }
90          return copia;
91      }
92  }
```

Líneas 81-92 Crea un tablero independiente con el mismo contenido. La IA lo usa para "probar" jugadas sin alterar la partida real.

Jugador.java original, sin cambios

Modelo (original) · 22 líneas · `Totito/src/Jugador.java`

Clase de datos: el nombre de un participante y la ficha con la que juega.

```
1  /**
2   * Representa al jugador humano: su nombre y la ficha que utiliza.
3   * Es una clase simple de datos, sin logica de juego.
4   */
5  public class Jugador {
6
```

Líneas 1-6 Declaración. No tiene lógica: solo guarda datos.

```
7      private final String nombre;
8      private final char ficha;
9
```

Líneas 7-9 Los dos campos son `final`: una vez creado el jugador, su nombre y su ficha no cambian. La pantalla de configuración crea un `Jugador` nuevo por cada partida.

```
10     public Jugador(String nombre, char ficha) {
11         this.nombre = nombre;
12         this.ficha = ficha;
13     }
14
```

Líneas 10-14 El constructor recibe los dos datos.

```
15     public String getNombre() {
16         return nombre;
17     }
18
19     public char getFicha() {
20         return ficha;
21     }
22 }
```

Líneas 15-22 Getters: la única forma de leer los campos privados. Se usan en el marcador, en "Turno de ..." y en "¡Ganó ...!".

IA_Minimax.java original, sin cambios

Inteligencia artificial (original) · 97 líneas · `Totito/src/IA_Minimax.java`

Decide la jugada de la computadora con el algoritmo Minimax. Recorre *todas* las partidas posibles a partir de la posición actual, así que nunca pierde.

```
1  /**
2   * Agente de Inteligencia Artificial que decide su jugada usando el
3   * algoritmo Minimax: explora recursivamente el arbol de todas las
4   * partidas posibles a partir del estado actual, asumiendo que la IA
5   * siempre maximiza su resultado y el jugador siempre lo minimiza.
6   * Como el Totito es un juego de suma cero con pocos estados posibles,
7   * el arbol se puede recorrer por completo, garantizando que la IA
8   * nunca pierda (gana o empata en todas las partidas).
9   */
10 public class IA_Minimax {
11
```

Líneas 1-11 El comentario de la clase explica la idea: la IA maximiza su puntaje y supone que el rival juega perfecto y lo minimiza.

```
12     private final char fichaIA;
13     private final char fichaJugador;
14
```

Líneas 12-14 Qué ficha usa la IA y cuál su rival. `Partida` los pasa en ese orden: `new IA_Minimax(fichaDelJugador2, fichaDelJugador1)`.

```
15     public IA_Minimax(char fichaIA, char fichaJugador) {
16         this.fichaIA = fichaIA;
17         this.fichaJugador = fichaJugador;
18     }
19
```

Líneas 15-19 Constructor: guarda las dos fichas.

```java
    /**
     * Evalua cada casilla vacia disponible, simula la jugada en una copia
     * del tablero y se queda con la de mayor puntaje segun Minimax.
     * Retorna {fila, columna} de la mejor jugada encontrada.
     */
    public int[] calcularMejorJugada(Tablero tablero) {
        int mejorValor = Integer.MIN_VALUE;
        int[] mejorJugada = new int[]{-1, -1};

        for (int fila = 0; fila < 3; fila++) {
            for (int columna = 0; columna < 3; columna++) {
                if (tablero.obtenerCasilla(fila, columna) == Tablero.VACIA) {
                    Tablero copia = tablero.obtenerCopia();
                    copia.marcarCasilla(fila, columna, fichaIA);
                    int valor = minimax(copia, 0, false);
                    if (valor > mejorValor) {
                        mejorValor = valor;
                        mejorJugada[0] = fila;
                        mejorJugada[1] = columna;
                    }
                }
            }
        }
        return mejorJugada;
    }

```

Líneas 20-45 Para cada casilla vacía: copia el tablero, pone ahí la ficha de la IA y pregunta a `minimax` qué tan buena es. Se queda con la de mayor puntaje y devuelve `{fila, columna}`. `Partida.jugarTurnoIA()` llama a este método.

```java
    /**
     * Funcion recursiva Minimax.
     * - Caso base: si el tablero ya tiene un ganador o esta lleno, retorna
     *   el puntaje de esa hoja del arbol (+10 si gana la IA, -10 si gana
     *   el jugador, 0 si es empate). Se resta/suma la profundidad para que
     *   la IA prefiera ganar rapido y perder (si fuera inevitable) tarde.
     * - Caso recursivo: prueba cada casilla vacia, simula la jugada del
     *   turno correspondiente y llama a minimax con el turno contrario.
     *   Si es turno de la IA (esMax = true) se queda con el valor MAXIMO
     *   entre las opciones; si es turno del jugador (esMax = false) se
     *   queda con el valor MINIMO, ya que se asume que el jugador tambien
     *   jugara de forma optima en su contra.
     */
    private int minimax(Tablero tablero, int profundidad, boolean esMax) {
        char ganador = tablero.verificarGanador();
        if (ganador == fichaIA) {
            return 10 - profundidad;
        }
        if (ganador == fichaJugador) {
            return profundidad - 10;
        }
        if (tablero.tableroLleno()) {
            return 0;
        }

```

Líneas 46-70 El método recursivo. **Casos base:** si gana la IA devuelve `10 - profundidad`; si gana el rival, `profundidad - 10`; si hay empate, 0. Restar la profundidad hace que prefiera ganar rápido y, si fuera a perder, perder tarde.

```
71          if (esMax) {
72              int mejor = Integer.MIN_VALUE;
73              for (int fila = 0; fila < 3; fila++) {
74                  for (int columna = 0; columna < 3; columna++) {
75                      if (tablero.obtenerCasilla(fila, columna) == Tablero.VACIA) {
76                          Tablero copia = tablero.obtenerCopia();
77                          copia.marcarCasilla(fila, columna, fichaIA);
78                          mejor = Math.max(mejor, minimax(copia, profundidad + 1, false));
79                      }
80                  }
81              }
82              return mejor;
```

Líneas 71-82 **Turno de la IA** (`esMax = true`): prueba cada casilla con su ficha y se queda con el valor **máximo**.

```
83          } else {
84              int menor = Integer.MAX_VALUE;
85              for (int fila = 0; fila < 3; fila++) {
86                  for (int columna = 0; columna < 3; columna++) {
87                      if (tablero.obtenerCasilla(fila, columna) == Tablero.VACIA) {
88                          Tablero copia = tablero.obtenerCopia();
89                          copia.marcarCasilla(fila, columna, fichaJugador);
90                          menor = Math.min(menor, minimax(copia, profundidad + 1, true));
91                      }
92                  }
93              }
94              return menor;
95          }
96      }
97  }
```

Líneas 83-97 **Turno del rival:** prueba cada casilla con la ficha del rival y se queda con el valor **mínimo**, porque supone que el rival juega lo mejor posible en su contra.

Parte B — Lógica nueva: Partida y Navegacion

`Partida` es el puente entre el código original y la interfaz. Es la única clase nueva con reglas del juego y no usa nada de Swing.

Partida.java nuevo

Modelo nuevo (controla la partida) · 160 líneas · `Totito/src/Partida.java`

Lleva el turno, el estado (en curso, victoria, empate), el ganador, el marcador acumulado y el modo de juego. Usa `Tablero`, `Jugador` e `IA_Minimax` sin modificarlos. Hace lo que en `JuegoMain` estaba mezclado con la ventana.

```
1  /**
2   * Una serie de partidas entre dos jugadores: el tablero, de quien es el
3   * turno, quien gano y el marcador acumulado.
4   *
5   * Reutiliza tal cual las clases originales del proyecto:
6   * - Tablero: guarda las casillas y sabe si hay ganador o empate.
7   * - Jugador: nombre y ficha de cada participante.
8   * - IA_Minimax: decide la jugada de la computadora en el modo contra la IA.
9   *
10  * No sabe nada de la interfaz grafica: la pantalla le pide jugar una
11  * casilla y despues le pregunta como quedo. Asi las reglas se pueden
12  * probar, o usar desde otra interfaz, sin abrir ninguna ventana.
13  */
14 public class Partida {
15
```

Líneas 1-15 El comentario de la clase enumera las tres clases originales que reutiliza. Que no sepa nada de la interfaz permite probar las reglas por programa, sin abrir ventanas.

```
16     /** Los dos modos que se eligen en la pantalla de configuracion. */
17     public enum Modo {
18         /** Dos personas se turnan en el mismo equipo. */
19         DOS_JUGADORES,
20         /** El Jugador 2 es la computadora (IA_Minimax). */
21         CONTRA_IA
22     }
23
```

Líneas 16-23 Un `enum` es un tipo con valores fijos. `DOS_JUGADORES` o `CONTRA_IA`: lo elige la pantalla de configuración.

```
24     /** Como va la partida actual. */
25     public enum Estado { EN_CURSO, VICTORIA, EMPATE }
26
```

Líneas 24-26 Cómo va la partida: `EN_CURSO`, `VICTORIA` o `EMPATE`. Las pantallas preguntan esto para saber qué mostrar.

```
27     /** Ficha que abre cada partida, como en el totito de siempre. */
28     public static final char FICHA_INICIAL = 'X';
29
```

Líneas 27-29 Siempre abre la X, como en el totito tradicional. Si quieren que empiece la O, basta con cambiar esta constante.

```
30      private final Tablero tablero = new Tablero();
31      private final Jugador[] jugadores;
32      private final Modo modo;
33      /** Solo existe en el modo contra la IA; la IA siempre es el Jugador 2. */
34      private final IA_Minimax ia;
35      private final int[] victorias = new int[2];
36      private int empates;
37
```

Líneas 30-37 El estado fijo de la serie: el tablero (original), los dos jugadores, el modo, la IA (que solo existe en modo IA) y el marcador. El marcador son las victorias por jugador y los empates; sobrevive a "Jugar de nuevo".

```
38      /** Indice (0 o 1) del jugador al que le toca. */
39      private int turno;
40      private Estado estado;
41      private Jugador ganador;
42
```

Líneas 38-42 El estado de la partida actual: a quién le toca (0 = Jugador 1, 1 = Jugador 2), cómo va y quién ganó.

```
43      public Partida(Jugador jugador1, Jugador jugador2, Modo modo) {
44          if (jugador1.getFicha() == jugador2.getFicha()) {
45              throw new IllegalArgumentException("Los dos jugadores no pueden usar la misma ficha");
46          }
47          this.jugadores = new Jugador[]{jugador1, jugador2};
48          this.modo = modo;
49          this.ia = modo == Modo.CONTRA_IA ? new IA_Minimax(jugador2.getFicha(), jugador1.getFicha()) : null;
50          reiniciar();
51      }
52
```

Líneas 43-52 Constructor. Rechaza que los dos tengan la misma ficha (lanza `IllegalArgumentException`). Solo crea la IA si el modo es `CONTRA_IA`, y siempre como Jugador 2. Termina llamando a `reiniciar()`.

```
53      /** Limpia el tablero para jugar otra vez. El marcador se conserva. */
54      public void reiniciar() {
55          tablero.reiniciar();
56          estado = Estado.EN_CURSO;
57          ganador = null;
58          turno = jugadores[0].getFicha() == FICHA_INICIAL ? 0 : 1;
59      }
60
```

Líneas 53-60 Limpia el tablero y deja el estado en curso. El primer turno es para quien tenga la `FICHA_INICIAL`: si el Jugador 1 eligió O, abre el Jugador 2. El marcador no se toca.

```
61      /**
62       * Coloca la ficha del jugador en turno en (fila, columna).
63       * Devuelve false si la jugada no vale (partida terminada o casilla
64       * ocupada); en ese caso no cambia nada.
65       */
66      public boolean jugar(int fila, int columna) {
67          if (estado != Estado.EN_CURSO) {
68              return false;
69          }
70          if (!tablero.marcarCasilla(fila, columna, jugadores[turno].getFicha())) {
71              return false;
72          }
73          if (tablero.verificarGanador() != Tablero.VACIA) {
74              estado = Estado.VICTORIA;
75              ganador = jugadores[turno];
76              victorias[turno]++;
77          } else if (tablero.tableroLleno()) {
78              estado = Estado.EMPATE;
79              empates++;
80          } else {
81              turno = 1 - turno;
82          }
83          return true;
84      }
85
```

Líneas 61-85 **El corazón del juego.** Si la partida terminó o la casilla está ocupada, devuelve `false` y no cambia nada. Si la jugada vale: con `verificarGanador()` mira si hubo victoria (suma al marcador) y con `tableroLleno()` si hubo empate. Si no pasó ninguna de las dos, `turno = 1 - turno` cambia de 0 a 1 o de 1 a 0.

```
86      /** true si la partida sigue y le toca mover a la computadora. */
87      public boolean esTurnoDeIA() {
88          return ia != null && turno == 1 && estado == Estado.EN_CURSO;
89      }
90
```

Líneas 86-90 True solo si hay IA, le toca al Jugador 2 y la partida sigue. `PantallaJuego` lo usa para bloquear los clics y programar la jugada de la computadora.

```
91      /** Le pide a IA_Minimax su mejor jugada y la aplica. */
92      public void jugarTurnoIA() {
93          if (!esTurnoDeIA()) {
94              return;
95          }
96          int[] jugada = ia.calcularMejorJugada(tablero);
97          jugar(jugada[0], jugada[1]);
98      }
99
```

Líneas 91-99 Le pide a `IA_Minimax` su mejor jugada y la aplica con el mismo `jugar()` que usa el humano. Así las reglas se aplican igual a los dos.

```
100     /** true si ese jugador es la computadora. */
101     public boolean esComputadora(Jugador jugador) {
102         return ia != null && jugador == jugadores[1];
103     }
104
```

Líneas 100-104 Dice si un jugador es la computadora. La pantalla de resultado lo usa para elegir la frase ("La máquina no perdona").

```
105     public Tablero getTablero() {
106         return tablero;
107     }
108
109     public Jugador getJugador(int indice) {
110         return jugadores[indice];
111     }
112
113     public Jugador getJugadorEnTurno() {
114         return jugadores[turno];
115     }
116
117     public Modo getModo() {
118         return modo;
119     }
120
121     public Estado getEstado() {
122         return estado;
123     }
124
125     /** El ganador de la ultima partida, o null si no hay (en curso o empate). */
126     public Jugador getGanador() {
127         return ganador;
128     }
129
130     public int getVictorias(int indice) {
131         return victorias[indice];
132     }
133
134     public int getEmpates() {
135         return empates;
136     }
137
```

Líneas 105-137 Getters de solo lectura para las pantallas.

```
138     /**
139      * Dice DONDE esta la linea ganadora, para que la interfaz dibuje la
140      * raya: {filaInicio, columnaInicio, filaFin, columnaFin}, o null si
141      * nadie ha ganado. Revisa lo mismo que Tablero.verificarGanador()
142      * (3 filas, 3 columnas y 2 diagonales), pero devolviendo posiciones.
143      */
144     public static int[] buscarLineaGanadora(Tablero tablero) {
145         int[][] lineas = {
146             {0, 0, 0, 1, 0, 2}, {1, 0, 1, 1, 1, 2}, {2, 0, 2, 1, 2, 2},   // filas
147             {0, 0, 1, 0, 2, 0}, {0, 1, 1, 1, 2, 1}, {0, 2, 1, 2, 2, 2},   // columnas
148             {0, 0, 1, 1, 2, 2}, {0, 2, 1, 1, 2, 0}                         // diagonales
149         };
150         for (int[] l : lineas) {
151             char a = tablero.obtenerCasilla(l[0], l[1]);
152             if (a != Tablero.VACIA
153                     && a == tablero.obtenerCasilla(l[2], l[3])
154                     && a == tablero.obtenerCasilla(l[4], l[5])) {
155                 return new int[]{l[0], l[1], l[4], l[5]};
156             }
157         }
158         return null;
159     }
160 }
```

Líneas 138-160 Hace la misma revisión que `verificarGanador()`, pero devuelve **dónde** está la línea: `{filaInicio, colInicio, filaFin, colFin}`. La tabla `lineas` tiene las 8 líneas posibles como tríos de coordenadas. Se puso aquí y no en `Tablero` para no modificar el código original.

Navegacion.java nuevo

Interfaz (contrato entre pantallas) · 26 líneas · `Totito/src/Navegacion.java`

Define lo que una pantalla puede pedir: ir a otra pantalla, empezar una partida, mostrar el resultado o salir.

```java
/**
 * Lo que las pantallas pueden pedir: cambiar a otra pantalla, empezar una
 * partida o cerrar el juego. Lo implementa PanelPrincipal.
 *
 * Gracias a esta interfaz ninguna pantalla conoce a las demas: solo dicen
 * "quiero ir al menu" y PanelPrincipal se encarga.
 */
public interface Navegacion {

    void irAlMenu();

    void irAInstrucciones();

    void irAConfiguracion();

```

Líneas 1-15 Una `interface` solo declara métodos, no los implementa. Las pantallas reciben un objeto `Navegacion` y lo llaman (`navegacion.irAlMenu()`) sin saber quién lo implementa. Lo implementa `PanelPrincipal`. Así ninguna pantalla depende de las demás.

```java
    /** Crea una partida nueva (marcador en cero) y muestra el tablero. */
    void iniciarPartida(Jugador jugador1, Jugador jugador2, Partida.Modo modo);

```

Líneas 16-18 Recibe los dos jugadores y el modo que se eligieron en la configuración.

```java
    /** Muestra la pantalla de victoria o empate de la partida terminada. */
    void mostrarResultado(Partida partida);

```

Líneas 19-21 Lo llama `PantallaJuego` cuando la partida termina.

```java
    /** Otra partida con los mismos jugadores, conservando el marcador. */
    void jugarDeNuevo();

    void salir();
}
```

Líneas 22-26 Otra partida con los mismos jugadores, conservando el marcador.

Parte C — Arranque y cambio de pantallas

`TotitoApp` abre la ventana y `PanelPrincipal` decide qué pantalla se ve.

TotitoApp.java nuevo

Punto de entrada (main) · 48 líneas · `Totito/src/TotitoApp.java`

La ventana del juego. Su `main` es lo que se ejecuta con el botón Run de NetBeans.

```
1  import java.awt.Graphics2D;
2  import java.awt.Image;
3  import java.awt.image.BufferedImage;
4  import java.awt.geom.RoundRectangle2D;
5  import javax.swing.JFrame;
6  import javax.swing.SwingUtilities;
7
```

Líneas 1-7 Imports de AWT/Swing que se necesitan: imagen para el icono, `JFrame` y `SwingUtilities`.

```
8  /**
9   * PUNTO DE ENTRADA de la version con interfaz neon (la del diseno).
10  * Para jugar, ejecutar el main de esta clase.
11  *
12  * La version original, con la ventana sencilla de Jugador contra IA,
13  * sigue en JuegoMain y tambien se puede ejecutar.
14  */
15 public class TotitoApp extends JFrame {
16
```

Líneas 8-16 Hereda de `JFrame`, la ventana de Swing. El comentario recuerda que la versión original (`JuegoMain`) sigue existiendo.

```
17     public TotitoApp() {
18         super("Juego del Totito");
19         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
20         setResizable(false);
21         setIconImage(crearIcono());
22         setContentPane(new PanelPrincipal(this::cerrar));
23         pack();
24         setLocationRelativeTo(null);   // centrada en la pantalla
25     }
26
```

Líneas 17-26 Configura la ventana: título; al cerrarla termina el programa; tamaño fijo (`setResizable(false)`, porque el diseño es de tamaño fijo); icono; y como contenido un `PanelPrincipal`. `this::cerrar` es una *referencia a método*: le pasa al panel "qué hacer al salir". `pack()` ajusta la ventana al tamaño del panel y `setLocationRelativeTo(null)` la centra.

```
27     private void cerrar() {
28         dispose();
29         System.exit(0);
30     }
31
```

Líneas 27-31 Cierra la ventana y termina el programa. Lo usan el botón Salir y la X del encabezado.

```
32      /** Icono de la ventana (barra de tareas): el logotipo sobre fondo oscuro. */
33      private static Image crearIcono() {
34          BufferedImage imagen = new BufferedImage(64, 64, BufferedImage.TYPE_INT_ARGB);
35          Graphics2D g2 = Dibujo.preparar(imagen.getGraphics());
36          g2.setColor(Tema.FONDO_ARRIBA);
37          g2.fill(new RoundRectangle2D.Double(0, 0, 64, 64, 16, 16));
38          Iconos.LOGO.dibujar(g2, 8, 8, 48, Tema.LAVANDA);
39          g2.dispose();
40          return imagen;
41      }
42
```

Líneas 32-42 Dibuja en una imagen de 64×64 el logotipo del juego. Es el icono que aparece en la barra de tareas; no hace falta ningún archivo .png.

```
43      public static void main(String[] args) {
44          // Texto suavizado tambien en Linux.
45          System.setProperty("awt.useSystemAAFontSettings", "on");
46          SwingUtilities.invokeLater(() -> new TotitoApp().setVisible(true));
47      }
48  }
```

Líneas 43-48 Arranque. Activa el texto suavizado en Linux y crea la ventana con `SwingUtilities.invokeLater`. Swing exige que toda la interfaz se cree y se toque desde su hilo (el *Event Dispatch Thread*), y `invokeLater` se encarga de eso.

PanelPrincipal.java nuevo

Controlador de pantallas · 91 líneas · `Totito/src/PanelPrincipal.java`

Contiene las 5 pantallas y muestra una a la vez con un `CardLayout`. Implementa `Navegacion` y guarda la `Partida` en curso.

```
1  import java.awt.CardLayout;
2  import java.awt.Dimension;
3  import javax.swing.JPanel;
4
```

Líneas 1-4 Imports: `CardLayout`, `Dimension` y `JPanel`.

```
5  /**
6   * Contiene todas las pantallas y muestra una a la vez (CardLayout, como
7   * un mazo de cartas). Implementa Navegacion: es quien cambia de pantalla
8   * cuando una de ellas lo pide, y quien guarda la Partida en curso.
9   *
10  * Es un JPanel y no la ventana, asi se puede probar o dibujar en una
11  * imagen sin abrir una ventana real.
12  */
13 public class PanelPrincipal extends JPanel implements Navegacion {
14
```

Líneas 5-14 Es un `JPanel` y no la ventana a propósito: así se puede crear sin pantalla física. Gracias a eso se generaron las capturas y se probó el flujo en el servidor.

```
15     /** Tamano de la ventana. Cabe en pantallas de 1366x768. */
16     public static final int ANCHO = 640;
17     public static final int ALTO = 690;
18
```

Líneas 15-18 Tamaño de la ventana: 640×690. Se eligió para que quepa en pantallas de 1366×768, comunes en laptops, contando la barra de título y la de tareas.

```
19     private static final String MENU = "menu";
20     private static final String INSTRUCCIONES = "instrucciones";
21     private static final String CONFIGURACION = "configuracion";
22     private static final String JUEGO = "juego";
23     private static final String RESULTADO = "resultado";
24
```

Líneas 19-24 Nombres de las "cartas": el `CardLayout` identifica cada pantalla por un texto.

```
25     private final CardLayout cartas = new CardLayout();
26     private final PantallaJuego pantallaJuego;
27     private final PantallaResultado pantallaResultado;
28     private final Runnable alSalir;
29     private Partida partida;
30
```

Líneas 25-30 El layout, las dos pantallas que hay que actualizar desde aquí (juego y resultado), la acción de salir y la partida actual.

```
31      /** @param alSalir que hacer al pulsar Salir o la X (cerrar la ventana). */
32      public PanelPrincipal(Runnable alSalir) {
33          this.alSalir = alSalir;
34          setLayout(cartas);
35          setPreferredSize(new Dimension(ANCHO, ALTO));
36
37          pantallaJuego = new PantallaJuego(this);
38          pantallaResultado = new PantallaResultado(this);
39          add(new PantallaMenu(this), MENU);
40          add(new PantallaInstrucciones(this), INSTRUCCIONES);
41          add(new PantallaConfiguracion(this), CONFIGURACION);
42          add(pantallaJuego, JUEGO);
43          add(pantallaResultado, RESULTADO);
44          cartas.show(this, MENU);
45      }
46
```

Líneas 31-46 Crea las 5 pantallas, pasándoles `this` como `Navegacion`, y las agrega al mazo con su nombre. Al final muestra el menú.

```
47      /** La partida en curso (null si aun no se ha empezado ninguna). */
48      public Partida getPartida() {
49          return partida;
50      }
51
```

Líneas 47-51 Da acceso a la partida en curso. Las pruebas automáticas lo usan.

```
52      @Override
53      public void irAlMenu() {
54          cartas.show(this, MENU);
55      }
56
57      @Override
58      public void irAInstrucciones() {
59          cartas.show(this, INSTRUCCIONES);
60      }
61
62      @Override
63      public void irAConfiguracion() {
64          cartas.show(this, CONFIGURACION);
65      }
66
```

Líneas 52-66 Cambiar de pantalla es una línea: `cartas.show(this, NOMBRE)`.

```
67      @Override
68      public void iniciarPartida(Jugador jugador1, Jugador jugador2, Partida.Modo modo) {
69          partida = new Partida(jugador1, jugador2, modo);
70          pantallaJuego.comenzar(partida);
71          cartas.show(this, JUEGO);
72      }
73
```

Líneas 67-73 Crea una `Partida` nueva (marcador en cero), se la da a la pantalla de juego y la muestra.

```
74      @Override
75      public void mostrarResultado(Partida partidaTerminada) {
76          pantallaResultado.mostrar(partidaTerminada);
77          cartas.show(this, RESULTADO);
78      }
79
```

Líneas 74-79 Prepara la pantalla de resultado con la partida terminada y la muestra.

```
80      @Override
81      public void jugarDeNuevo() {
82          partida.reiniciar();
83          pantallaJuego.comenzar(partida);
84          cartas.show(this, JUEGO);
85      }
86
```

Líneas 80-86 Reinicia la misma partida (conserva marcador y jugadores) y vuelve al tablero.

```
87      @Override
88      public void salir() {
89          alSalir.run();
90      }
91  }
```

Líneas 87-91 Ejecuta la acción de salir que le pasó `TotitoApp`.

Parte D — El estilo: colores, dibujo e iconos

Tres clases de utilidades con métodos `static`. Todo lo visual sale de aquí, así que para cambiar el aspecto del juego no hace falta tocar las pantallas.

Tema.java nuevo

Paleta de colores y fuentes · 103 líneas · `Totito/src/Tema.java`

Todos los colores y fuentes de la interfaz en un solo lugar.

```
1  import java.awt.Color;
2  import java.awt.Font;
3  import java.awt.GraphicsEnvironment;
4  import java.awt.font.TextAttribute;
5  import java.util.Arrays;
6  import java.util.List;
7  import java.util.Map;
8
```

Líneas 1-8 Imports: colores, fuentes y atributos de texto (para el espaciado de letras).

```
9  /**
10  * Paleta de colores y tipografia de la interfaz "neon".
11  *
12  * Toda la interfaz toma sus colores y fuentes de aqui: para cambiar el
13  * aspecto del juego basta con tocar esta clase, no hay colores sueltos
14  * repartidos por las pantallas.
15  */
16 public final class Tema {
17
```

Líneas 9-17 `final`: no se puede heredar. Solo tiene constantes y métodos estáticos.

```
18     /** Fondo de la ventana: degradado de arriba hacia abajo. */
19     public static final Color FONDO_ARRIBA = new Color(0x1C, 0x17, 0x40);
20     public static final Color FONDO_ABAJO = new Color(0x0E, 0x0B, 0x24);
21     /** X y O gigantes y desvaidas que adornan el fondo. */
22     public static final Color DECORACION = new Color(0x5B, 0x35, 0xC2, 70);
23
```

Líneas 18-23 Colores del fondo: degradado de morado oscuro a casi negro. `DECORACION` tiene alfa 70 (semitransparente) para las X y O gigantes del fondo.

```
24     /** Color de la X (en todo el juego la X es rosa). */
25     public static final Color ROSA = new Color(0xFF, 0x3B, 0xD4);
26     /** Tarjeta del Jugador 1: un rosa mas rojizo, como en el diseno. */
27     public static final Color FRESA = new Color(0xFF, 0x3D, 0x82);
28     /** Color de la O (en todo el juego la O es azul). */
29     public static final Color AZUL = new Color(0x2F, 0x84, 0xFF);
30     public static final Color CIAN = new Color(0x40, 0xC8, 0xFF);
31     public static final Color MORADO = new Color(0xA0, 0x5C, 0xFF);
32     public static final Color LAVANDA = new Color(0xC8, 0xBC, 0xF6);
33
```

Líneas 24-33 Los colores neón del diseño. Regla de todo el juego: **la X es ROSA y la O es AZUL**.

```
34      public static final Color TEXTO = Color.WHITE;
35      public static final Color TEXTO_SUAVE = new Color(0xA8, 0x9B, 0xE2);
36      /** Relleno de las tarjetas "de cristal": semitransparente a proposito. */
37      public static final Color SUPERFICIE = new Color(0x2B, 0x23, 0x63, 125);
38      /** Fondo de cada casilla del tablero. */
39      public static final Color CASILLA = new Color(0x2E, 0x28, 0x5A, 170);
40
```

Líneas 34-40 Colores de texto y superficies. `SUPERFICIE` es semitransparente (alfa 125), y eso da el efecto "cristal": a través de las tarjetas se ve un poco el fondo.

```
41      /**
42       * Fuentes preferidas, en orden. Se usa la primera que exista en el
43       * equipo (Windows suele tener Segoe UI; Linux, DejaVu o Liberation).
44       * Si no hay ninguna, Java usa "SansSerif", que existe siempre.
45       */
46      private static final List<String> FAMILIAS = Arrays.asList(
47              "Nunito", "Poppins", "Montserrat", "Segoe UI", "Helvetica Neue",
48              "Ubuntu", "Arial", "Liberation Sans", "DejaVu Sans");
49
```

Líneas 41-49 Lista de fuentes en orden de preferencia. Java no trae la fuente redondeada del diseño, así que se usa la mejor que tenga el equipo: Segoe UI en Windows, DejaVu o Liberation en Linux.

```
50      private static final String FAMILIA = elegirFamilia();
51
52      private Tema() {
53      }
54
```

Líneas 50-54 Se elige la fuente una sola vez, al cargar la clase.

```
55      /** Fuente de la interfaz. estilo = Font.PLAIN o Font.BOLD. */
56      public static Font fuente(int estilo, float tamano) {
57          return new Font(FAMILIA, estilo, 12).deriveFont(tamano);
58      }
59
```

Líneas 55-59 Crea la fuente de la interfaz en el estilo (normal o negrita) y tamaño pedidos.

```
60      /** Fuente con las letras separadas, para titulos tipo "J U G A D O R E S". */
61      public static Font fuenteEspaciada(int estilo, float tamano, float espaciado) {
62          return fuente(estilo, tamano).deriveFont(Map.of(TextAttribute.TRACKING, espaciado));
63      }
64
```

Líneas 60-64 Fuente con `TextAttribute.TRACKING`: separa las letras, como en "J U G A D O R E S".

```
65      /** La X siempre es rosa y la O siempre es azul, la juegue quien la juegue. */
66      public static Color colorDeFicha(char ficha) {
67          return ficha == 'X' ? ROSA : AZUL;
68      }
69
```

Líneas 65-69 Traduce una ficha a su color. Todas las clases lo usan para que la regla rosa/azul se cumpla en todas partes.

```
70      /** El mismo color con otra transparencia (0 = invisible, 255 = opaco). */
71      public static Color conAlfa(Color color, int alfa) {
72          return new Color(color.getRed(), color.getGreen(), color.getBlue(),
73                  Math.max(0, Math.min(255, alfa)));
74      }
75
```

Líneas 70-75 Devuelve el mismo color con otra transparencia (0 = invisible, 255 = opaco). `Math.max/min` evita valores fuera de rango.

```
76      /** Mezcla el color con blanco: factor 0 lo deja igual, 1 lo vuelve blanco. */
77      public static Color aclarar(Color color, float factor) {
78          int r = Math.round(color.getRed() + (255 - color.getRed()) * factor);
79          int g = Math.round(color.getGreen() + (255 - color.getGreen()) * factor);
80          int b = Math.round(color.getBlue() + (255 - color.getBlue()) * factor);
81          return new Color(r, g, b, color.getAlpha());
82      }
83
```

Líneas 76-83 Mezcla el color con blanco: cada componente se acerca a 255 según el factor. Sirve para los degradados y el "centro blanco" del neón.

```
84      /** "#RRGGBB", para usar los colores del tema dentro de texto HTML de un JLabel. */
85      public static String hex(Color color) {
86          return String.format("#%02X%02X%02X", color.getRed(), color.getGreen(), color.getBlue());
87      }
88
```

Líneas 84-88 Convierte un color a texto "#RRGGBB". Se usa para colorear la X y la O dentro del HTML de las instrucciones.

```
89      private static String elegirFamilia() {
90          try {
91              List<String> instaladas = Arrays.asList(
92                      GraphicsEnvironment.getLocalGraphicsEnvironment().getAvailableFontFamilyNames());
93              for (String familia : FAMILIAS) {
94                  if (instaladas.contains(familia)) {
95                      return familia;
96                  }
97              }
98          } catch (RuntimeException e) {
99              // Sin acceso a las fuentes del sistema: se usa la generica.
100         }
101         return Font.SANS_SERIF;
102     }
103 }
```

Líneas 89-103 Pregunta al sistema qué fuentes hay instaladas y devuelve la primera de la lista que exista. Si algo falla, usa "SansSerif", que Java garantiza que existe.

Dibujo.java nuevo

Utilidades de dibujo (el efecto neón) · 166 líneas · `Totito/src/Dibujo.java`

Funciones compartidas: preparar el Graphics, el trazo neón, las fichas X/O y el texto con halo.

```
1  import java.awt.BasicStroke;
2  import java.awt.Color;
3  import java.awt.Dimension;
4  import java.awt.FontMetrics;
5  import java.awt.Graphics;
6  import java.awt.Graphics2D;
7  import java.awt.Paint;
8  import java.awt.RenderingHints;
9  import java.awt.Shape;
10 import java.awt.Stroke;
11 import java.awt.geom.Ellipse2D;
12 import java.awt.geom.Path2D;
13 import javax.swing.JComponent;
14
```

Líneas 1–14 Imports de Java2D (formas, trazos, colores) y de Swing.

```
15 /**
16  * Herramientas de dibujo que comparten todos los componentes de la interfaz.
17  *
18  * El efecto "neon" es un truco sencillo: se traza la misma figura varias
19  * veces, primero muy gruesa y casi transparente, y cada vez mas fina y mas
20  * opaca. El ojo lo interpreta como luz que se difumina alrededor del trazo.
21  * No se usan imagenes: todo (fichas, iconos, botones) se dibuja con Java2D.
22  */
23 public final class Dibujo {
24
25     private Dibujo() {
26     }
27
```

Líneas 15–27 El comentario explica el truco del neón, en el que se basa toda la estética del juego.

```
28     /**
29      * Devuelve una copia del Graphics con el suavizado (antialiasing)
30      * activado. Quien la pida debe llamar dispose() al terminar.
31      */
32     public static Graphics2D preparar(Graphics g) {
33         Graphics2D g2 = (Graphics2D) g.create();
34         g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
35         g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
36         g2.setRenderingHint(RenderingHints.KEY_RENDERING, RenderingHints.VALUE_RENDER_QUALITY);
37         g2.setRenderingHint(RenderingHints.KEY_STROKE_CONTROL, RenderingHints.VALUE_STROKE_PURE);
38         g2.setRenderingHint(RenderingHints.KEY_FRACTIONALMETRICS, RenderingHints.VALUE_FRACTIONALMETRICS_ON);
39         return g2;
40     }
41
```

Líneas 28–41 Copia el `Graphics` y activa el suavizado (antialiasing) para líneas y texto. Sin esto los bordes se verían dentados. Se trabaja sobre una copia (`g.create()`) para no alterar el Graphics que Swing reutiliza; por eso cada componente termina con `g2.dispose()` .

```
42      /**
43       * Fija el tamano de un componente (preferido, minimo y maximo iguales)
44       * para que los BoxLayout de las pantallas no lo estiren.
45       */
46      public static void fijarTamano(JComponent componente, int ancho, int alto) {
47          Dimension tamano = new Dimension(ancho, alto);
48          componente.setPreferredSize(tamano);
49          componente.setMinimumSize(tamano);
50          componente.setMaximumSize(tamano);
51      }
52
```

Líneas 42-52 Pone el mismo tamaño preferido, mínimo y máximo. Los `BoxLayout` estiran los componentes hasta su tamaño máximo; fijándolo, cada pieza conserva el tamaño del diseño.

```
53      /** Trazo con puntas y esquinas redondeadas. */
54      public static BasicStroke redondo(float grosor) {
55          return new BasicStroke(grosor, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND);
56      }
57
```

Líneas 53-57 Trazo con puntas y uniones redondeadas, como los tubos de neón.

```
58      /** Trazo simple, sin halo. */
59      public static void trazo(Graphics2D g, Shape figura, Color color, float grosor) {
60          g.setColor(color);
61          g.setStroke(redondo(grosor));
62          g.draw(figura);
63      }
64
```

Líneas 58-64 Trazo simple, sin halo, para detalles pequeños.

```
65      public static void trazoNeon(Graphics2D g, Shape figura, Color color, float grosor) {
66          trazoNeon(g, figura, color, grosor, 1f);
67      }
68
```

Líneas 65-68 Versión corta de `trazoNeon` con intensidad normal (1).

```
69      /**
70       * Traza la figura con halo de neon.
71       *
72       * @param intensidad 1 = normal; menos de 1 se ve apagado (opcion no
73       *                   elegida); mas de 1 brilla mas (raton encima).
74       */
75      public static void trazoNeon(Graphics2D g, Shape figura, Color color, float grosor, float intensidad) {
76          Stroke trazoPrevio = g.getStroke();
77          Paint pinturaPrevia = g.getPaint();
78
```

Líneas 69-78 **El efecto neón.** Guarda el trazo y la pintura actuales para dejarlos como estaban al final.

```
79          // Halo: de la capa mas ancha y transparente a la mas fina y opaca.
80          final int capas = 6;
81          for (int i = capas; i >= 1; i--) {
82              int alfa = Math.round((6 + (capas - i) * 5) * intensidad);
83              g.setColor(Tema.conAlfa(color, alfa));
84              g.setStroke(redondo(grosor + i * 2.4f));
85              g.draw(figura);
86          }
87
```

Líneas 79-87 Dibuja la figura 6 veces: la primera muy gruesa (grosor + 14.4 px) y casi transparente (alfa 6); cada vuelta es más fina y más opaca. Al superponerse se forma un halo que se difumina hacia afuera. `intensidad` multiplica el alfa: con menos de 1 se ve apagado (opción no elegida), con más de 1 brilla más (ratón encima).

```
88          // Linea principal.
89          g.setColor(Tema.conAlfa(color, Math.round(255 * Math.min(1f, 0.4f + 0.6f * intensidad))));
90          g.setStroke(redondo(grosor));
91          g.draw(figura);
92
```

Líneas 88-92 La línea nítida del grosor pedido. Con poca intensidad también se atenúa.

```
93          // En trazos gruesos (las fichas) el centro del "tubo" es casi blanco.
94          if (grosor >= 5) {
95              g.setColor(Tema.conAlfa(Tema.aclarar(color, 0.6f), 200));
96              g.setStroke(redondo(grosor * 0.35f));
97              g.draw(figura);
98          }
99
100         g.setStroke(trazoPrevio);
101         g.setPaint(pinturaPrevia);
102     }
103
```

Líneas 93-103 En trazos de 5 px o más (las fichas) se agrega un centro casi blanco, como el de un tubo de neón real. Luego se restauran el trazo y la pintura.

```
104     /** Dibuja una X de neon centrada en (cx, cy). */
105     public static void dibujarX(Graphics2D g, double cx, double cy, double tam, Color color, float grosor) {
106         double r = tam / 2;
107         Path2D equis = new Path2D.Double();
108         equis.moveTo(cx - r, cy - r);
109         equis.lineTo(cx + r, cy + r);
110         equis.moveTo(cx + r, cy - r);
111         equis.lineTo(cx - r, cy + r);
112         trazoNeon(g, equis, color, grosor);
113     }
114
```

Líneas 104-114 Una X son dos diagonales en un `Path2D` (moveTo/lineTo), trazadas con neón.

```
115     /** Dibuja una O de neon centrada en (cx, cy). */
116     public static void dibujarO(Graphics2D g, double cx, double cy, double tam, Color color, float grosor) {
117         trazoNeon(g, new Ellipse2D.Double(cx - tam / 2, cy - tam / 2, tam, tam), color, grosor);
118     }
119
```

Líneas 115-119 Una O es un círculo (`Ellipse2D`) trazado con neón.

```
120     /**
121      * Dibuja la ficha ('X' u 'O') con su color del tema. La X se hace un
122      * poco mas pequena para que a la vista ocupe lo mismo que la O.
123      */
124     public static void dibujarFicha(Graphics2D g, char ficha, double cx, double cy, double tam, float grosor) {
125         if (ficha == 'X') {
126             dibujarX(g, cx, cy, tam * 0.84, Tema.ROSA, grosor);
127         } else if (ficha == 'O') {
128             dibujarO(g, cx, cy, tam, Tema.AZUL, grosor);
129         }
130     }
131
```

Líneas 120-131 Dibuja `'X'` u `'O'` con su color. La X va al 84 % del tamaño porque sus diagonales la hacen ver más grande que un círculo del mismo ancho.

```
132     /**
133      * Escribe texto con un halo de color alrededor.
134      * (x, y) es la linea base del texto, como en drawString.
135      */
136     public static void textoNeon(Graphics2D g, String texto, float x, float y, Color halo, Paint relleno) {
137         Paint previo = g.getPaint();
138         for (int radio = 4; radio >= 1; radio--) {
139             g.setColor(Tema.conAlfa(halo, 4 + (4 - radio) * 3));
140             for (int angulo = 0; angulo < 360; angulo += 45) {
141                 double rad = Math.toRadians(angulo);
142                 g.drawString(texto, x + (float) (Math.cos(rad) * radio), y + (float) (Math.sin(rad) * radio));
143             }
144         }
145         g.setPaint(relleno);
146         g.drawString(texto, x, y);
147         g.setPaint(previo);
148     }
149
```

Líneas 132-149 Texto con halo: lo escribe en 8 direcciones (cada 45°) y a 4 distancias, con color muy transparente, y encima el texto real con su relleno (que puede ser un degradado).

```
150     /** Linea base para que el texto quede centrado en vertical sobre yCentro. */
151     public static float lineaBaseCentrada(FontMetrics fm, float yCentro) {
152         return yCentro + (fm.getAscent() - fm.getDescent()) / 2f;
153     }
154
```

Líneas 150-154 `drawString` recibe la *línea base* del texto, no su centro. Esta fórmula calcula la línea base para que el texto quede centrado en vertical.

```
155     /** Recorta el texto con "…" si no cabe en el ancho dado. */
156     public static String recortar(FontMetrics fm, String texto, int anchoMaximo) {
157         if (fm.stringWidth(texto) <= anchoMaximo) {
158             return texto;
159         }
160         String recortado = texto;
161         while (recortado.length() > 1 && fm.stringWidth(recortado + "…") > anchoMaximo) {
162             recortado = recortado.substring(0, recortado.length() - 1);
163         }
164         return recortado + "…";
165     }
166 }
```

Líneas 155-166 Si un nombre no cabe, le va quitando letras hasta que quepa con "…".

Iconos.java nuevo

Iconos dibujados con código · 268 líneas · `Totito/src/Iconos.java`

Cada icono del diseño (play, libro, casa, trofeo...) hecho con figuras geométricas. No hay archivos de imagen.

```
1  import java.awt.Color;
2  import java.awt.GradientPaint;
3  import java.awt.Graphics2D;
4  import java.awt.Shape;
5  import java.awt.geom.Arc2D;
6  import java.awt.geom.Area;
7  import java.awt.geom.Ellipse2D;
8  import java.awt.geom.Path2D;
9  import java.awt.geom.Rectangle2D;
10 import java.awt.geom.RoundRectangle2D;
11
```

Líneas 1-11 Imports de figuras de Java2D: arcos, áreas, elipses, rutas, rectángulos.

```
12 /**
13  * Iconos de la interfaz, dibujados con figuras geometricas (sin archivos
14  * de imagen). Cada icono se dibuja dentro de un cuadrado de lado "t" cuya
15  * esquina superior izquierda es (x, y); todas las medidas son fracciones
16  * de t, asi el mismo icono sirve a cualquier tamano.
17  *
18  * Para crear uno nuevo basta con agregar otra constante siguiendo el mismo
19  * patron.
20  */
21 public final class Iconos {
22
```

Líneas 12-22 Regla común: cada icono se dibuja en un cuadrado de lado `t` que empieza en (x, y), y todas las medidas son fracciones de `t` (por ejemplo, `t * 0.5` es la mitad). Así el mismo icono sirve a 20 px o a 84 px.

```
23     /** Un icono sabe dibujarse en el cuadrado (x, y, tam, tam) con el color dado. */
24     @FunctionalInterface
25     public interface Icono {
26         void dibujar(Graphics2D g, double x, double y, double tam, Color color);
27     }
28
```

Líneas 23-28 Interfaz funcional (un solo método). Gracias a eso cada icono se escribe como una *lambda*: `(g, x, y, t, c) -> { ... }`. Los botones reciben un `Icono` y lo dibujan sin saber cuál es.

```
29     private Iconos() {
30     }
31
```

Líneas 29-31 Constructor privado: nadie crea objetos `Iconos`, solo se usan sus constantes.

```
32      /** Triangulo de "reproducir" (botones Jugar y Comenzar partida). */
33      public static final Icono JUGAR = (g, x, y, t, c) -> {
34          Path2D triangulo = new Path2D.Double();
35          triangulo.moveTo(x + t * 0.24, y + t * 0.14);
36          triangulo.lineTo(x + t * 0.86, y + t * 0.5);
37          triangulo.lineTo(x + t * 0.24, y + t * 0.86);
38          triangulo.closePath();
39          Dibujo.trazoNeon(g, triangulo, c, (float) (t * 0.1), 0.6f);
40          g.setPaint(new GradientPaint((float) x, (float) y, Tema.aclarar(c, 0.25f),
41                  (float) (x + t), (float) (y + t), c));
42          g.fill(triangulo);
43      };
44
```

Líneas 32-44 Triángulo de "reproducir": tres puntos y `closePath()`. Primero un halo neón y luego relleno con un degradado diagonal (`GradientPaint`).

```
45      /** Libro abierto (Instrucciones / Como jugar). */
46      public static final Icono LIBRO = (g, x, y, t, c) -> {
47          double cx = x + t / 2;
48          double arriba = y + t * 0.2;
49          double abajo = y + t * 0.84;
50          Path2D libro = new Path2D.Double();
51          // Pagina izquierda
52          libro.moveTo(cx, arriba + t * 0.06);
53          libro.quadTo(cx - t * 0.2, arriba - t * 0.06, x + t * 0.06, arriba + t * 0.02);
54          libro.lineTo(x + t * 0.06, abajo - t * 0.08);
55          libro.quadTo(cx - t * 0.2, abajo - t * 0.14, cx, abajo);
56          // Pagina derecha
57          libro.moveTo(cx, arriba + t * 0.06);
58          libro.quadTo(cx + t * 0.2, arriba - t * 0.06, x + t * 0.94, arriba + t * 0.02);
59          libro.lineTo(x + t * 0.94, abajo - t * 0.08);
60          libro.quadTo(cx + t * 0.2, abajo - t * 0.14, cx, abajo);
61          // Lomo
62          libro.moveTo(cx, arriba + t * 0.06);
63          libro.lineTo(cx, abajo);
64          Dibujo.trazoNeon(g, libro, c, (float) (t * 0.075));
65      };
66
```

Líneas 45-66 Libro abierto: cada página es una curva (`quadTo`) arriba, una línea al costado y otra curva abajo, más una línea al centro (el lomo).

```
67      /** Puerta con flecha saliendo (Salir). */
68      public static final Icono SALIR = (g, x, y, t, c) -> {
69          Path2D puerta = new Path2D.Double();
70          puerta.moveTo(x + t * 0.55, y + t * 0.3);
71          puerta.lineTo(x + t * 0.55, y + t * 0.12);
72          puerta.lineTo(x + t * 0.14, y + t * 0.12);
73          puerta.lineTo(x + t * 0.14, y + t * 0.88);
74          puerta.lineTo(x + t * 0.55, y + t * 0.88);
75          puerta.lineTo(x + t * 0.55, y + t * 0.7);
76          puerta.moveTo(x + t * 0.36, y + t * 0.5);
77          puerta.lineTo(x + t * 0.9, y + t * 0.5);
78          puerta.moveTo(x + t * 0.74, y + t * 0.34);
79          puerta.lineTo(x + t * 0.9, y + t * 0.5);
80          puerta.lineTo(x + t * 0.74, y + t * 0.66);
81          Dibujo.trazoNeon(g, puerta, c, (float) (t * 0.075));
82      };
83
```

Líneas 67-83 Puerta (línea en forma de corchete) más una flecha que sale hacia la derecha.

```
84      /** Flechita "›" del lado derecho de los botones del menu. */
85      public static final Icono CHEVRON = (g, x, y, t, c) -> {
86          Path2D flecha = new Path2D.Double();
87          flecha.moveTo(x + t * 0.32, y + t * 0.16);
88          flecha.lineTo(x + t * 0.68, y + t * 0.5);
89          flecha.lineTo(x + t * 0.32, y + t * 0.84);
90          Dibujo.trazoNeon(g, flecha, c, (float) (t * 0.13), 0.7f);
91      };
92
```

Líneas 84-92 La flechita "›" de los botones del menú: dos segmentos en ángulo.

```
93      /** Flecha hacia la izquierda (Volver / Regresar). */
94      public static final Icono FLECHA_IZQUIERDA = (g, x, y, t, c) -> {
95          Path2D flecha = new Path2D.Double();
96          flecha.moveTo(x + t * 0.88, y + t * 0.5);
97          flecha.lineTo(x + t * 0.14, y + t * 0.5);
98          flecha.moveTo(x + t * 0.42, y + t * 0.22);
99          flecha.lineTo(x + t * 0.14, y + t * 0.5);
100         flecha.lineTo(x + t * 0.42, y + t * 0.78);
101         Dibujo.trazo(g, flecha, c, (float) (t * 0.1));
102     };
103
```

Líneas 93-103 Flecha "←" para Volver y Regresar: una línea horizontal y una punta.

```
104     /** Casa (Menu principal). */
105     public static final Icono CASA = (g, x, y, t, c) -> {
106         g.setColor(c);
107         Path2D techo = new Path2D.Double();
108         techo.moveTo(x + t * 0.5, y + t * 0.1);
109         techo.lineTo(x + t * 0.92, y + t * 0.48);
110         techo.lineTo(x + t * 0.08, y + t * 0.48);
111         techo.closePath();
112         g.setStroke(Dibujo.redondo((float) (t * 0.08)));
113         g.draw(techo);
114         g.fill(techo);
115         Area cuerpo = new Area(new Rectangle2D.Double(x + t * 0.2, y + t * 0.44, t * 0.6, t * 0.46));
116         cuerpo.subtract(new Area(new RoundRectangle2D.Double(
117                 x + t * 0.41, y + t * 0.6, t * 0.18, t * 0.32, t * 0.05, t * 0.05)));
118         g.fill(cuerpo);
119     };
120
```

Líneas 104-120 Casa: triángulo del techo relleno. Cuerpo: un rectángulo al que se le *resta* la puerta con `Area.subtract`, que deja un hueco recortado.

```
121     /** Dos flechas en circulo (Jugar de nuevo / Reiniciar). */
122     public static final Icono REINICIAR = (g, x, y, t, c) -> {
123         double cx = x + t / 2;
124         double cy = y + t / 2;
125         double r = t * 0.34;
126         g.setColor(c);
127         g.setStroke(Dibujo.redondo((float) (t * 0.1)));
128         for (int inicio : new int[]{30, 210}) {
129             g.draw(new Arc2D.Double(cx - r, cy - r, 2 * r, 2 * r, inicio, 120, Arc2D.OPEN));
```

Líneas 121-129 Dos arcos (`Arc2D`) de 120° cada uno, opuestos entre sí.

```
            // Punta de flecha al final del arco, apuntando en el sentido del giro.
            double a = Math.toRadians(inicio + 120);
            double px = cx + r * Math.cos(a);
            double py = cy - r * Math.sin(a);
            double tx = -Math.sin(a);   // direccion tangente (sentido antihorario)
            double ty = -Math.cos(a);
            double nx = Math.cos(a);    // direccion radial
            double ny = -Math.sin(a);
            double largo = t * 0.17;
            double ancho = t * 0.14;
            Path2D punta = new Path2D.Double();
            punta.moveTo(px + tx * largo, py + ty * largo);
            punta.lineTo(px + nx * ancho, py + ny * ancho);
            punta.lineTo(px - nx * ancho, py - ny * ancho);
            punta.closePath();
            g.fill(punta);
        }
    };

```

Líneas 130-148 Punta de flecha al final de cada arco. Con seno y coseno del ángulo final se calcula el punto donde termina el arco, la dirección tangente (hacia dónde gira) y la radial (perpendicular). Con eso se arma un triángulo que apunta en el sentido del giro.

```
    /** Silueta de persona (tarjetas de jugador). */
    public static final Icono PERSONA = Iconos::persona;

```

Líneas 149-151 `Iconos::persona` es una referencia al método privado de más abajo.

```
    /** Dos personas (modo 2 jugadores). */
    public static final Icono PERSONAS = (g, x, y, t, c) -> {
        persona(g, x + t * 0.42, y + t * 0.26, t * 0.52, Tema.conAlfa(c, 170));
        persona(g, x, y + t * 0.12, t * 0.66, c);
    };

```

Líneas 152-157 Dos personas: una pequeña y semitransparente detrás y otra grande delante.

```
    /** Chip de computadora (modo contra la IA). */
    public static final Icono COMPUTADORA = (g, x, y, t, c) -> {
        float grosor = (float) (t * 0.07);
        Dibujo.trazo(g, new RoundRectangle2D.Double(x + t * 0.2, y + t * 0.2, t * 0.6, t * 0.6, t * 0.16, t * 0.16),
                c, grosor);
        g.fill(new RoundRectangle2D.Double(x + t * 0.37, y + t * 0.37, t * 0.26, t * 0.26, t * 0.06, t * 0.06));
        Path2D patas = new Path2D.Double();
        for (int i = 0; i < 3; i++) {
            double p = t * (0.34 + 0.16 * i);
            patas.moveTo(x + p, y + t * 0.05);
            patas.lineTo(x + p, y + t * 0.19);
            patas.moveTo(x + p, y + t * 0.81);
            patas.lineTo(x + p, y + t * 0.95);
            patas.moveTo(x + t * 0.05, y + p);
            patas.lineTo(x + t * 0.19, y + p);
            patas.moveTo(x + t * 0.81, y + p);
            patas.lineTo(x + t * 0.95, y + p);
        }
        Dibujo.trazo(g, patas, c, grosor);
    };

```

Líneas 158-178 Chip: un cuadrado redondeado, un cuadrado relleno adentro y tres "patitas" por lado, hechas con un `for`.

```
    /** Trofeo con estrella (pantalla de victoria). */
    public static final Icono TROFEO = (g, x, y, t, c) -> {
        Path2D copa = new Path2D.Double();
        copa.moveTo(x + t * 0.24, y + t * 0.08);
        copa.lineTo(x + t * 0.76, y + t * 0.08);
        copa.lineTo(x + t * 0.73, y + t * 0.36);
        copa.quadTo(x + t * 0.69, y + t * 0.58, x + t * 0.5, y + t * 0.6);
        copa.quadTo(x + t * 0.31, y + t * 0.58, x + t * 0.27, y + t * 0.36);
        copa.closePath();

        Path2D asas = new Path2D.Double();
        asas.moveTo(x + t * 0.25, y + t * 0.16);
        asas.lineTo(x + t * 0.1, y + t * 0.16);
        asas.quadTo(x + t * 0.08, y + t * 0.42, x + t * 0.31, y + t * 0.46);
        asas.moveTo(x + t * 0.75, y + t * 0.16);
        asas.lineTo(x + t * 0.9, y + t * 0.16);
        asas.quadTo(x + t * 0.92, y + t * 0.42, x + t * 0.69, y + t * 0.46);

        Area pie = new Area(new Rectangle2D.Double(x + t * 0.45, y + t * 0.58, t * 0.1, t * 0.16));
        pie.add(new Area(new RoundRectangle2D.Double(x + t * 0.33, y + t * 0.72, t * 0.34, t * 0.08, t * 0.04, t * 0.04)));
        pie.add(new Area(new RoundRectangle2D.Double(x + t * 0.26, y + t * 0.8, t * 0.48, t * 0.11, t * 0.05, t * 0.05)));

        Dibujo.trazoNeon(g, copa, c, 1.5f, 0.9f);
        Dibujo.trazoNeon(g, asas, c, (float) (t * 0.06));
        g.setPaint(new GradientPaint(0, (float) y, Tema.aclarar(c, 0.35f), 0, (float) (y + t), c));
        g.fill(copa);
        g.fill(pie);
        g.setColor(Tema.conAlfa(Tema.FONDO_ABAJO, 190));
        g.fill(estrella(x + t * 0.5, y + t * 0.32, t * 0.11, t * 0.045));
    };

```

Líneas 179-209 Trofeo: la copa es una ruta con curvas, las asas son dos curvas trazadas y el pie es la unión (`Area.add`) de tres rectángulos. Se rellena con degradado vertical y se le pone una estrella oscura encima.

```
    /** Logotipo del encabezado: la rejilla del totito con dos O y una X. */
    public static final Icono LOGO = (g, x, y, t, c) -> {
        float grosor = (float) (t * 0.045);
        Path2D rejilla = new Path2D.Double();
        rejilla.moveTo(x + t * 0.37, y + t * 0.04);
        rejilla.lineTo(x + t * 0.35, y + t * 0.96);
        rejilla.moveTo(x + t * 0.65, y + t * 0.04);
        rejilla.lineTo(x + t * 0.63, y + t * 0.96);
        rejilla.moveTo(x + t * 0.04, y + t * 0.36);
        rejilla.lineTo(x + t * 0.96, y + t * 0.34);
        rejilla.moveTo(x + t * 0.04, y + t * 0.66);
        rejilla.lineTo(x + t * 0.96, y + t * 0.64);
        Dibujo.trazoNeon(g, rejilla, c, grosor, 0.6f);
        Dibujo.trazoNeon(g, new Ellipse2D.Double(x + t * 0.08, y + t * 0.08, t * 0.18, t * 0.18), Tema.MORADO, grosor, 0.6f);
        Dibujo.trazoNeon(g, new Ellipse2D.Double(x + t * 0.74, y + t * 0.07, t * 0.18, t * 0.18), Tema.MORADO, grosor, 0.6f);
        Path2D equis = new Path2D.Double();
        equis.moveTo(x + t * 0.09, y + t * 0.75);
        equis.lineTo(x + t * 0.25, y + t * 0.91);
        equis.moveTo(x + t * 0.25, y + t * 0.75);
        equis.lineTo(x + t * 0.09, y + t * 0.91);
        Dibujo.trazoNeon(g, equis, Tema.MORADO, grosor, 0.6f);
    };

```

Líneas 210-232 Logotipo del encabezado: la rejilla "#" con líneas ligeramente inclinadas, como dibujada a mano, dos O arriba y una X abajo.

```
233     /** X de cerrar (esquina del encabezado). */
234     public static final Icono CERRAR = (g, x, y, t, c) -> {
235         Path2D equis = new Path2D.Double();
236         equis.moveTo(x + t * 0.22, y + t * 0.22);
237         equis.lineTo(x + t * 0.78, y + t * 0.78);
238         equis.moveTo(x + t * 0.78, y + t * 0.22);
239         equis.lineTo(x + t * 0.22, y + t * 0.78);
240         Dibujo.trazoNeon(g, equis, c, (float) (t * 0.11), 0.8f);
241     };
242
```

Líneas 233-242 La X del botón de cerrar.

```
243     private static void persona(Graphics2D g, double x, double y, double t, Color c) {
244         g.setPaint(new GradientPaint(0, (float) y, Tema.aclarar(c, 0.25f), 0, (float) (y + t), c));
245         g.fill(new Ellipse2D.Double(x + t * 0.3, y + t * 0.04, t * 0.4, t * 0.4));
246         Area cuerpo = new Area(new Ellipse2D.Double(x + t * 0.08, y + t * 0.5, t * 0.84, t * 0.84));
247         cuerpo.intersect(new Area(new Rectangle2D.Double(x, y + t * 0.5, t, t * 0.46)));
248         g.fill(cuerpo);
249     }
250
```

Líneas 243-250 Silueta: cabeza (círculo) y cuerpo. El cuerpo es un círculo grande cortado por abajo con `Area.intersect`; queda una cúpula de base plana.

```
251     /** Estrella de cinco puntas centrada en (cx, cy). */
252     private static Shape estrella(double cx, double cy, double radioExterior, double radioInterior) {
253         Path2D estrella = new Path2D.Double();
254         for (int i = 0; i < 10; i++) {
255             double r = i % 2 == 0 ? radioExterior : radioInterior;
256             double a = -Math.PI / 2 + i * Math.PI / 5;
257             double px = cx + Math.cos(a) * r;
258             double py = cy + Math.sin(a) * r;
259             if (i == 0) {
260                 estrella.moveTo(px, py);
261             } else {
262                 estrella.lineTo(px, py);
263             }
264         }
265         estrella.closePath();
266         return estrella;
267     }
268 }
```

Líneas 251-268 Estrella de 5 puntas: 10 vértices que alternan radio exterior e interior, cada 36°.

Parte E — Componentes reutilizables

Piezas de interfaz que se dibujan a mano en `paintComponent`. El patrón es siempre el mismo: `Graphics2D g2 = Dibujo.preparar(g)`, dibujar con formas de Java2D y terminar con `g2.dispose()`.

Tarjeta.java nuevo

Panel "de cristal" · 56 líneas · `Totito/src/Tarjeta.java`

Caja redondeada semitransparente con borde neón. Envuelve el menú, las instrucciones, las tarjetas de jugador y el resultado.

```
1  import java.awt.Color;
2  import java.awt.Graphics;
3  import java.awt.Graphics2D;
4  import java.awt.geom.RoundRectangle2D;
5  import javax.swing.JPanel;
6
```

Líneas 1-6 Imports.

```
7  /**
8   * Panel "de cristal": rectangulo redondeado semitransparente con borde de
9   * neon. Es la caja que envuelve el menu, las instrucciones, las tarjetas de
10  * jugador y el resultado. Se le agregan componentes como a cualquier JPanel.
11  *
12  * Las subclases pueden sobrescribir pintarDecoracion() para dibujar algo
13  * encima del fondo y debajo de los componentes (por ejemplo, confeti).
14  */
15 public class Tarjeta extends JPanel {
16
```

Líneas 7-16 Hereda de `JPanel`: se le agregan componentes como a cualquier panel.

```
17     /** Espacio entre el borde del componente y la caja, para que quepa el halo. */
18     public static final int MARGEN = 10;
19
```

Líneas 17-19 10 px entre el borde del componente y la caja. Son necesarios porque el halo se dibuja *por fuera* de la caja, y lo que cae fuera del componente no se pinta.

```
20     private final Color color;
21     private final int arco;
22
```

Líneas 20-22 Color del borde y radio de las esquinas.

```
23      public Tarjeta(Color color) {
24          this(color, 30);
25      }
26
27      public Tarjeta(Color color, int arco) {
28          this.color = color;
29          this.arco = arco;
30          setOpaque(false);
31          setAlignmentX(CENTER_ALIGNMENT);
32      }
33
```

Líneas 23-33 Constructores. `setOpaque(false)` le dice a Swing que el panel es transparente, así que debe pintar lo que hay detrás antes que él.

```
34      /** La caja redondeada, ya descontado el margen del halo. */
35      protected RoundRectangle2D forma() {
36          return new RoundRectangle2D.Double(MARGEN, MARGEN,
37                  getWidth() - 2.0 * MARGEN, getHeight() - 2.0 * MARGEN, arco, arco);
38      }
39
```

Líneas 34-39 La caja redondeada, ya descontado el margen.

```
40      @Override
41      protected void paintComponent(Graphics g) {
42          Graphics2D g2 = Dibujo.preparar(g);
43          RoundRectangle2D forma = forma();
44          g2.setColor(Tema.SUPERFICIE);
45          g2.fill(forma);
46          g2.setColor(Tema.conAlfa(color, 12));
47          g2.fill(forma);
48          pintarDecoracion(g2, forma);
49          Dibujo.trazoNeon(g2, forma, color, 1.8f);
50          g2.dispose();
51      }
52
```

Líneas 40-52 Pinta la superficie semitransparente, una tinta muy suave del color, la decoración opcional y el borde neón. Swing llama a este método cada vez que hay que redibujar.

```
53      /** Gancho para las subclases. Por defecto no dibuja nada. */
54      protected void pintarDecoracion(Graphics2D g2, RoundRectangle2D forma) {
55      }
56  }
```

Líneas 53-56 "Gancho" para subclases: por defecto no hace nada. `PantallaResultado` lo sobrescribe para dibujar el confeti.

BotonNeon.java nuevo

Botón con estilo neón · 192 líneas · `Totito/src/BotonNeon.java`

Un `JButton` real (con `addActionListener`, teclado, foco) dibujado a mano. Tiene 4 estilos.

```
1  import java.awt.AlphaComposite;
2  import java.awt.BasicStroke;
3  import java.awt.Color;
4  import java.awt.Cursor;
5  import java.awt.Font;
6  import java.awt.FontMetrics;
7  import java.awt.GradientPaint;
8  import java.awt.Graphics;
9  import java.awt.Graphics2D;
10 import java.awt.geom.Line2D;
11 import java.awt.geom.RoundRectangle2D;
12 import javax.swing.JButton;
13
```

Líneas 1-13 Imports.

```
14 /**
15  * Boton con borde de neon. Es un JButton normal (se usa con
16  * addActionListener), solo que se dibuja a mano.
17  *
18  * Se crea con los metodos de fabrica, uno por cada forma del diseno:
19  * menu(...), pildora(...), contorno(...) e icono(...).
20  */
21 public class BotonNeon extends JButton {
22
```

Líneas 14-22 Hereda de `JButton`: conserva todo el comportamiento de un botón y solo cambia su apariencia.

```
23     /** Las formas de boton que aparecen en el diseno. */
24     public enum Estilo {
25         /** Boton ancho del menu principal:  icono | Texto   › */
26         MENU,
27         /** Pildora rellena con degradado ("Comenzar partida", "Jugar de nuevo"). */
28         PILDORA,
29         /** Pildora solo con borde ("Volver", "Reiniciar"). */
30         CONTORNO,
31         /** Cuadrado pequeno con solo un icono (la X de cerrar). */
32         ICONO
33     }
34
```

Líneas 23-34 Las 4 formas del diseño: MENU (ancho, con icono, divisor y flecha), PILDORA (rellena con degradado), CONTORNO (solo borde) e ICONO (cuadrado pequeño).

```
35     /** Espacio alrededor de la forma para que el halo no se corte. */
36     private static final int MARGEN = 9;
37
```

Líneas 35-37 Espacio para el halo, igual que en `Tarjeta`.

```
38     private final Estilo estilo;
39     private final Iconos.Icono icono;
40     private final Color color;
41     private final Color colorFinal;
42
```

Líneas 38-42 Datos del botón: estilo, icono y los dos colores del degradado.

```
43      private BotonNeon(String texto, Iconos.Icono icono, Color color, Color colorFinal,
44                        Estilo estilo, int ancho, int alto) {
45          super(texto);
46          this.estilo = estilo;
47          this.icono = icono;
48          this.color = color;
49          this.colorFinal = colorFinal;
50
```

Líneas 43-50 Constructor **privado**: los botones se crean con los métodos de fábrica de abajo. Así cada estilo trae su tamaño por defecto y no se pueden combinar parámetros que no tienen sentido.

```
51          // Se apaga todo lo que Swing dibuja por defecto: lo pinta paintComponent.
52          setContentAreaFilled(false);
53          setBorderPainted(false);
54          setFocusPainted(false);
55          setOpaque(false);
56          setRolloverEnabled(true);
57
58          setForeground(Tema.TEXTO);
59          setFont(Tema.fuente(Font.BOLD, estilo == Estilo.MENU ? 25f : 19f));
60          setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
61          setAlignmentX(CENTER_ALIGNMENT);
62          Dibujo.fijarTamano(this, ancho, alto);
63      }
64
```

Líneas 51-64 Se apaga todo lo que Swing pinta por defecto (fondo, borde, recuadro de foco) porque lo dibuja `paintComponent`. `setRolloverEnabled(true)` permite saber si el ratón está encima. También se ponen el texto blanco, la fuente, la mano como cursor, el centrado y el tamaño fijo.

```
65      /** Boton grande del menu principal. */
66      public static BotonNeon menu(String texto, Iconos.Icono icono, Color color) {
67          return new BotonNeon(texto, icono, color, color, Estilo.MENU, 384, 88);
68      }
69
```

Líneas 65-69 Fábrica del botón del menú principal (384×88).

```
70      /** Pildora rellena con un degradado de "desde" a "hasta". */
71      public static BotonNeon pildora(String texto, Iconos.Icono icono, Color desde, Color hasta, int ancho, int a
    lto) {
72          return new BotonNeon(texto, icono, desde, hasta, Estilo.PILDORA, ancho, alto);
73      }
74
```

Líneas 70-74 Fábrica de la píldora rellena; el que la llama decide los colores y el tamaño.

```
75      /** Pildora oscura con borde morado. */
76      public static BotonNeon contorno(String texto, Iconos.Icono icono, int ancho, int alto) {
77          return new BotonNeon(texto, icono, Tema.MORADO, Tema.MORADO, Estilo.CONTORNO, ancho, alto);
78      }
79
```

Líneas 75-79 Fábrica de la píldora con borde morado.

```
80      /** Boton cuadrado sin texto. Ponerle setToolTipText para que se entienda. */
81      public static BotonNeon icono(Iconos.Icono icono, Color color) {
82          return new BotonNeon("", icono, color, color, Estilo.ICONO, 58, 58);
83      }
84
```

Líneas 80-84 Fábrica del botón cuadrado sin texto (la X de cerrar).

```
85      @Override
86      protected void paintComponent(Graphics g) {
87          Graphics2D g2 = Dibujo.preparar(g);
88          if (!isEnabled()) {
89              g2.setComposite(AlphaComposite.getInstance(AlphaComposite.SRC_OVER, 0.4f));
90          }
91          boolean resaltado = getModel().isRollover() && isEnabled();
92          boolean pulsado = getModel().isArmed() && getModel().isPressed();
93
94          double ancho = getWidth() - 2.0 * MARGEN;
95          double alto = getHeight() - 2.0 * MARGEN;
```

Líneas 85-95 Si el botón está deshabilitado, todo se dibuja al 40 % de opacidad (`AlphaComposite`). Se lee el estado del modelo del botón: *rollover* (ratón encima) y *pressed* (apretado).

```
96          double arco = switch (estilo) {
97              case MENU -> 30;
98              case ICONO -> 16;
99              default -> alto;        // píldora: extremos totalmente redondos
100         };
101         RoundRectangle2D forma = new RoundRectangle2D.Double(MARGEN, MARGEN, ancho, alto, arco, arco);
102
```

Líneas 96-102 Un `switch` de expresión (Java 14+) elige el redondeo: en las píldoras el arco es igual a la altura, y por eso los extremos quedan semicirculares.

```
103         pintarRelleno(g2, forma, resaltado);
104         Color borde = estilo == Estilo.PILDORA ? Tema.aclarar(color, 0.2f) : color;
105         Dibujo.trazoNeon(g2, forma, borde, 2f, resaltado ? 1.5f : 1f);
106         if (pulsado) {
107             g2.setColor(new Color(0, 0, 0, 60));
108             g2.fill(forma);
109         }
110
111         if (estilo == Estilo.MENU) {
112             pintarContenidoMenu(g2, forma);
113         } else {
114             pintarContenidoCentrado(g2, forma);
115         }
116         g2.dispose();
117     }
118
```

Líneas 103-118 Orden de pintado: relleno, borde neón (más intenso con el ratón encima), una sombra si está apretado y el contenido según el estilo.

```
119     private void pintarRelleno(Graphics2D g2, RoundRectangle2D forma, boolean resaltado) {
120         float x0 = (float) forma.getX();
121         float x1 = (float) (forma.getX() + forma.getWidth());
122         switch (estilo) {
123             case MENU -> g2.setPaint(new GradientPaint(
124                     x0, 0, Tema.conAlfa(color, resaltado ? 95 : 65),
125                     x1, 0, Tema.conAlfa(Tema.FONDO_ABAJO, 120)));
126             case PILDORA -> g2.setPaint(new GradientPaint(
127                     x0, 0, Tema.conAlfa(color, resaltado ? 255 : 225),
128                     x1, 0, Tema.conAlfa(colorFinal, resaltado ? 255 : 225)));
129             default -> g2.setPaint(Tema.conAlfa(color, resaltado ? 60 : 30));
130         }
131         g2.fill(forma);
132
133         if (estilo == Estilo.PILDORA) {
```

Líneas 119-133 Relleno según el estilo: el menú usa un degradado horizontal del color a oscuro, la píldora un degradado entre sus dos colores y los demás una tinta translúcida.

```
            // Reflejo de "cristal" en la mitad de arriba.
            float y0 = (float) forma.getY();
            g2.setPaint(new GradientPaint(0, y0, new Color(255, 255, 255, 50),
                    0, y0 + (float) forma.getHeight() / 2, new Color(255, 255, 255, 0)));
            g2.fill(forma);
        }
    }
```

Líneas 134-141 En las píldoras se agrega un reflejo blanco que se desvanece en la mitad de arriba; da sensación de cristal.

```
    /** Distribucion del menu: icono a la izquierda, divisor, texto y flechita. */
    private void pintarContenidoMenu(Graphics2D g2, RoundRectangle2D forma) {
        double x = forma.getX();
        double y = forma.getY();
        double ancho = forma.getWidth();
        double alto = forma.getHeight();

        double tamIcono = alto * 0.5;
        double divisor = x + ancho * 0.27;
        icono.dibujar(g2, x + ancho * 0.135 - tamIcono / 2, y + (alto - tamIcono) / 2, tamIcono,
                Tema.aclarar(color, 0.15f));

        g2.setColor(new Color(255, 255, 255, 45));
        g2.setStroke(new BasicStroke(1.5f));
        g2.draw(new Line2D.Double(divisor, y + alto * 0.22, divisor, y + alto * 0.78));

        g2.setFont(getFont());
        g2.setColor(getForeground());
        FontMetrics fm = g2.getFontMetrics();
        g2.drawString(getText(), (float) (divisor + 30), Dibujo.lineaBaseCentrada(fm, (float) (y + alto / 2)));

        double tamFlecha = alto * 0.3;
        Iconos.CHEVRON.dibujar(g2, x + ancho - 28 - tamFlecha, y + (alto - tamFlecha) / 2, tamFlecha,
                Tema.aclarar(color, 0.2f));
    }
```

Líneas 142-167 Distribución del botón del menú: icono centrado en el primer 27 % del ancho, una línea divisoria, el texto alineado a la izquierda y la flechita a la derecha.

```
168     /** Distribucion de pildoras e iconos: icono + texto centrados como un bloque. */
169     private void pintarContenidoCentrado(Graphics2D g2, RoundRectangle2D forma) {
170         double alto = forma.getHeight();
171         double cy = forma.getY() + alto / 2;
172         String texto = getText();
173 
174         if (texto == null || texto.isEmpty()) {
175             double t = alto * 0.5;
176             icono.dibujar(g2, forma.getCenterX() - t / 2, cy - t / 2, t, color);
177             return;
178         }
179 
180         g2.setFont(getFont());
181         FontMetrics fm = g2.getFontMetrics();
182         double tamIcono = icono == null ? 0 : alto * 0.44;
183         double hueco = icono == null ? 0 : 16;
184         double total = tamIcono + hueco + fm.stringWidth(texto);
185         double x = forma.getCenterX() - total / 2;
186         if (icono != null) {
187             icono.dibujar(g2, x, cy - tamIcono / 2, tamIcono, Tema.TEXTO);
188         }
189         g2.setColor(getForeground());
190         g2.drawString(texto, (float) (x + tamIcono + hueco), Dibujo.lineaBaseCentrada(fm, (float) cy));
191     }
192 }
```

Líneas 168-192 Para píldoras e iconos: si no hay texto, solo el icono. Si hay texto, se mide el ancho de icono + espacio + texto y se centra el conjunto.

Encabezado.java nuevo

Barra superior · 77 líneas · `Totito/src/Encabezado.java`

Logotipo, título "Juego del Totito" con degradado, lema y botón de cerrar. Aparece en todas las pantallas.

```
1  import java.awt.Dimension;
2  import java.awt.Font;
3  import java.awt.FontMetrics;
4  import java.awt.GradientPaint;
5  import java.awt.Graphics;
6  import java.awt.Graphics2D;
7  import java.awt.geom.RoundRectangle2D;
8  import javax.swing.JPanel;
9
```

Líneas 1-9 Imports.

```
10 /**
11  * Barra superior comun a todas las pantallas: logotipo, el titulo
12  * "Juego del Totito", el lema y el boton de cerrar.
13  */
14 public class Encabezado extends JPanel {
15
```

Líneas 10-15 Es un `JPanel` porque contiene un componente real: el botón de cerrar.

```
16     public static final int ALTO = 96;
17     private static final int MARGEN = 10;
18     private static final int LADO_BOTON = 58;
19
20     private final BotonNeon botonCerrar;
21
```

Líneas 16-21 Alto fijo de 96 px. El ancho lo pone el `BorderLayout` de la pantalla.

```
22     /** @param alCerrar que hacer al pulsar la X (normalmente, cerrar el juego). */
23     public Encabezado(Runnable alCerrar) {
24         super(null);    // sin layout: el unico hijo (el boton) se coloca en doLayout()
25         setOpaque(false);
26         setPreferredSize(new Dimension(0, ALTO));   // el ancho lo decide el BorderLayout
27
28         botonCerrar = BotonNeon.icono(Iconos.CERRAR, Tema.ROSA);
29         botonCerrar.setToolTipText("Cerrar el juego");
30         botonCerrar.addActionListener(e -> alCerrar.run());
31         add(botonCerrar);
32     }
33
```

Líneas 22-33 `super(null)` crea el panel **sin layout**: el botón se coloca a mano. `Runnable` es "una acción sin parámetros": la pantalla decide qué pasa al cerrar.

```
34     @Override
35     public void doLayout() {
36         botonCerrar.setBounds(getWidth() - MARGEN - LADO_BOTON - 6, (getHeight() - LADO_BOTON) / 2,
37                 LADO_BOTON, LADO_BOTON);
38     }
39
```

Líneas 34-39 Swing llama a `doLayout` cuando cambia el tamaño. Aquí se pone el botón a la derecha y centrado en vertical.

```
40      @Override
41      protected void paintComponent(Graphics g) {
42          Graphics2D g2 = Dibujo.preparar(g);
43          int ancho = getWidth();
44
```

Líneas 40-44 Dibujo del encabezado.

```
45          // Caja translucida con borde suave.
46          RoundRectangle2D caja = new RoundRectangle2D.Double(MARGEN, MARGEN,
47                  ancho - 2.0 * MARGEN, getHeight() - 2.0 * MARGEN, 26, 26);
48          g2.setPaint(new GradientPaint(0, MARGEN, Tema.conAlfa(Tema.LAVANDA, 22),
49                  0, getHeight() - MARGEN, Tema.conAlfa(Tema.LAVANDA, 8)));
50          g2.fill(caja);
51          Dibujo.trazoNeon(g2, caja, Tema.MORADO, 1.2f, 0.45f);
52
```

Líneas 45-52 La caja con degradado lavanda muy tenue y un borde neón suave (intensidad 0.45).

```
53          double ladoLogo = 52;
54          Iconos.LOGO.dibujar(g2, MARGEN + 20, (getHeight() - ladoLogo) / 2, ladoLogo, Tema.LAVANDA);
55
```

Líneas 53-55 El logotipo a la izquierda.

```
56          // Titulo: "Juego del " en blanco y "Totito" con degradado rosa -> cian.
57          g2.setFont(Tema.fuente(Font.BOLD, 36f));
58          FontMetrics fm = g2.getFontMetrics();
59          String parte1 = "Juego del ";
60          String parte2 = "Totito";
61          float anchoTitulo = fm.stringWidth(parte1) + fm.stringWidth(parte2);
62          float x = (ancho - anchoTitulo) / 2f;
63          float base = MARGEN + 44;
64          Dibujo.textoNeon(g2, parte1, x, base, Tema.LAVANDA, Tema.TEXTO);
65          float x2 = x + fm.stringWidth(parte1);
66          Dibujo.textoNeon(g2, parte2, x2, base, Tema.ROSA,
67                  new GradientPaint(x2, 0, Tema.ROSA, x2 + fm.stringWidth(parte2), 0, Tema.CIAN));
68
```

Líneas 56-68 El título en dos partes: se mide el ancho total para centrarlo, "Juego del " va en blanco y "Totito" con degradado de rosa a cian, las dos con halo.

```
69          // Lema con letras separadas.
70          g2.setFont(Tema.fuenteEspaciada(Font.BOLD, 10.5f, 0.28f));
71          fm = g2.getFontMetrics();
72          String lema = "ESTRATEGIA  •  DIVERSIÓN  •  SIEMPRE UN CLÁSICO";
73          g2.setColor(Tema.TEXTO_SUAVE);
74          g2.drawString(lema, (ancho - fm.stringWidth(lema)) / 2f, base + 24);
75          g2.dispose();
76      }
77  }
```

Líneas 69-77 El lema en mayúsculas espaciadas y color suave.

Rotulo.java nuevo

Textos decorativos · 166 líneas · `Totito/src/Rotulo.java`

Títulos con degradado ("Cómo jugar", "¡Ganó Ana!") y títulos de sección entre líneas ("— JUGADORES —").

```
1  import java.awt.BasicStroke;
2  import java.awt.Color;
3  import java.awt.Font;
4  import java.awt.FontMetrics;
5  import java.awt.GradientPaint;
6  import java.awt.Graphics;
7  import java.awt.Graphics2D;
8  import java.awt.geom.Line2D;
9  import javax.swing.JComponent;
10
```

Líneas 1-10 Imports.

```
11 /**
12  * Texto decorativo centrado. Hay dos variantes, que se crean con:
13  *
14  * - Rotulo.titulo(...): texto grande con degradado y halo, opcionalmente
15  *   con icono a la izquierda y subrayado ("Cómo jugar", "¡Ganó Ana!").
16  *   Si el texto no cabe, la letra se achica sola.
17  * - Rotulo.seccion(...): texto entre dos lineas que se desvanecen, con un
18  *   subtitulo opcional ("— JUGADORES —", "— Excelente jugada —").
19  */
20 public class Rotulo extends JComponent {
21
```

Líneas 11-21 El comentario resume las variantes.

```
22     private enum Tipo { TITULO, SECCION }
23
```

Líneas 22-23 Tipo interno: TITULO o SECCION. Decide qué método de pintado se usa.

```
24     private final Tipo tipo;
25     private final Font fuente;
26     private String texto;
27     private String subtitulo;
28     private Color desde;
29     private Color hasta;
30     private Iconos.Icono icono;
31     private Color colorIcono;
32     private Color subrayadoDesde;
33     private Color subrayadoHasta;
34
```

Líneas 24-34 Datos: tipo, fuente, texto, subtítulo, colores del degradado, icono y subrayado opcionales.

```
35     private Rotulo(Tipo tipo, String texto, Font fuente, Color desde, Color hasta, int ancho, int alto) {
36         this.tipo = tipo;
37         this.texto = texto;
38         this.fuente = fuente;
39         this.desde = desde;
40         this.hasta = hasta;
41         setAlignmentX(CENTER_ALIGNMENT);
42         Dibujo.fijarTamano(this, ancho, alto);
43     }
44
```

Líneas 35-44 Constructor privado; se usan las fábricas.

```
45      /** Titulo grande con degradado de color "desde" a "hasta". */
46      public static Rotulo titulo(String texto, float tamano, Color desde, Color hasta, int ancho, int alto) {
47          return new Rotulo(Tipo.TITULO, texto, Tema.fuente(Font.BOLD, tamano), desde, hasta, ancho, alto);
48      }
49
```

Líneas 45-49 Título grande en negrita con degradado de `desde` a `hasta`.

```
50      /** Titulo de seccion en mayusculas espaciadas, con subtitulo opcional (puede ser null). */
51      public static Rotulo seccion(String texto, String subtitulo, int ancho) {
52          Rotulo r = new Rotulo(Tipo.SECCION, texto, Tema.fuenteEspaciada(Font.BOLD, 17f, 0.3f),
53                  Tema.LAVANDA, Tema.LAVANDA, ancho, subtitulo == null ? 28 : 48);
54          r.subtitulo = subtitulo;
55          return r;
56      }
57
```

Líneas 50-57 Título de sección en mayúsculas espaciadas, con subtítulo opcional. El alto cambia según haya subtítulo o no.

```
58      /** Frase corta entre lineas, sin mayusculas espaciadas ("Excelente jugada"). */
59      public static Rotulo leyenda(String texto, int ancho) {
60          return new Rotulo(Tipo.SECCION, texto, Tema.fuente(Font.BOLD, 19f),
61                  Tema.LAVANDA, Tema.LAVANDA, ancho, 32);
62      }
63
```

Líneas 58-63 Frase corta entre líneas, sin espaciado ("Excelente jugada").

```
64      /** Agrega un icono a la izquierda del titulo. */
65      public Rotulo conIcono(Iconos.Icono icono, Color color) {
66          this.icono = icono;
67          this.colorIcono = color;
68          return this;
69      }
70
71      /** Agrega un subrayado con degradado bajo el titulo. */
72      public Rotulo conSubrayado(Color desde, Color hasta) {
73          this.subrayadoDesde = desde;
74          this.subrayadoHasta = hasta;
75          return this;
76      }
77
```

Líneas 64-77 Métodos "fluidos": devuelven `this` para encadenar `Rotulo.titulo(...).conIcono(...).conSubrayado(...)`.

```
78      public void setTexto(String texto) {
79          this.texto = texto;
80          repaint();
81      }
82
83      public void setTexto(String texto, Color desde, Color hasta) {
84          this.desde = desde;
85          this.hasta = hasta;
86          setTexto(texto);
87      }
88
```

Líneas 78-88 Cambiar el texto (y opcionalmente los colores) y pedir que se redibuje. `repaint()` no dibuja de inmediato: le avisa a Swing que lo haga en cuanto pueda.

```
89      @Override
90      protected void paintComponent(Graphics g) {
91          Graphics2D g2 = Dibujo.preparar(g);
92          if (tipo == Tipo.TITULO) {
93              pintarTitulo(g2);
94          } else {
95              pintarSeccion(g2);
96          }
97          g2.dispose();
98      }
99
```

Líneas 89-99 Según el tipo, llama a uno de los dos métodos de pintado.

```
100     private void pintarTitulo(Graphics2D g2) {
101         boolean conSubrayado = subrayadoDesde != null;
102         Font f = fuente;
103         g2.setFont(f);
104         FontMetrics fm = g2.getFontMetrics();
105
```

Líneas 100-105 Título. Primero se elige la fuente...

```
106         // Si no cabe, se reduce la letra hasta que quepa.
107         double extraIcono = icono == null ? 0 : fm.getAscent() + 16;
108         while (fm.stringWidth(texto) + extraIcono > getWidth() - 20 && f.getSize2D() > 12) {
109             f = f.deriveFont(f.getSize2D() - 1);
110             g2.setFont(f);
111             fm = g2.getFontMetrics();
112             extraIcono = icono == null ? 0 : fm.getAscent() + 16;
113         }
114
```

Líneas 106-114 ...y si el texto es más ancho que el componente, se achica la letra de 1 en 1 hasta que quepa. Así un nombre largo en "¡Ganó ...!" no se sale.

```
115         int anchoTexto = fm.stringWidth(texto);
116         double tamIcono = icono == null ? 0 : fm.getAscent() * 1.05;
117         double hueco = icono == null ? 0 : 16;
118         double x = (getWidth() - (tamIcono + hueco + anchoTexto)) / 2;
119         float base = Dibujo.lineaBaseCentrada(fm, getHeight() / 2f - (conSubrayado ? 6 : 0));
120
121         if (icono != null) {
122             icono.dibujar(g2, x, base - fm.getAscent() * 0.95, tamIcono, colorIcono);
123         }
124         float xTexto = (float) (x + tamIcono + hueco);
125         Dibujo.textoNeon(g2, texto, xTexto, base, desde,
126                 new GradientPaint(xTexto, 0, desde, xTexto + anchoTexto, 0, hasta));
127
```

Líneas 115-127 Se centra el bloque icono + texto, se dibuja el icono y luego el texto con halo y degradado horizontal.

```
128         if (conSubrayado) {
129             float y = base + 14;
130             g2.setPaint(new GradientPaint(xTexto, 0, subrayadoDesde, xTexto + anchoTexto, 0, subrayadoHasta));
131             g2.setStroke(Dibujo.redondo(3f));
132             g2.draw(new Line2D.Float(xTexto, y, xTexto + anchoTexto, y));
133         }
134     }
135
```

Líneas 128-135 Subrayado opcional con degradado, debajo del texto.

```
136     private void pintarSeccion(Graphics2D g2) {
137         g2.setFont(fuente);
138         FontMetrics fm = g2.getFontMetrics();
139         int anchoTexto = fm.stringWidth(texto);
140         float x = (getWidth() - anchoTexto) / 2f;
141         float base = subtitulo == null ? Dibujo.lineaBaseCentrada(fm, getHeight() / 2f) : 21;
142         Dibujo.textoNeon(g2, texto, x, base, Tema.MORADO, desde);
143
```

Líneas 136-143 Sección: el texto centrado con halo morado.

```
144         // Lineas a los lados que se desvanecen hacia afuera.
145         float yLinea = base - fm.getAscent() * 0.35f;
146         float separacion = 24;
147         // 140 px, o menos si no caben dentro del componente.
148         float largo = Math.min(140, (getWidth() - anchoTexto) / 2f - separacion - 16);
149         Color linea = Tema.conAlfa(Tema.MORADO, 170);
150         Color nada = Tema.conAlfa(Tema.MORADO, 0);
151         g2.setStroke(new BasicStroke(1.6f));
152         float finIzquierda = x - separacion;
153         g2.setPaint(new GradientPaint(finIzquierda - largo, 0, nada, finIzquierda, 0, linea));
154         g2.draw(new Line2D.Float(finIzquierda - largo, yLinea, finIzquierda, yLinea));
155         float inicioDerecha = x + anchoTexto + separacion;
156         g2.setPaint(new GradientPaint(inicioDerecha, 0, linea, inicioDerecha + largo, 0, nada));
157         g2.draw(new Line2D.Float(inicioDerecha, yLinea, inicioDerecha + largo, yLinea));
158
```

Líneas 144-158 Dos líneas que se desvanecen hacia afuera: un degradado de transparente a morado a la izquierda y de morado a transparente a la derecha. El largo se limita para no tocar los bordes.

```
159         if (subtitulo != null) {
160             g2.setFont(Tema.fuente(Font.PLAIN, 14f));
161             fm = g2.getFontMetrics();
162             g2.setColor(Tema.TEXTO_SUAVE);
163             g2.drawString(subtitulo, (getWidth() - fm.stringWidth(subtitulo)) / 2f, base + 22);
164         }
165     }
166 }
```

Líneas 159-166 Subtítulo opcional en letra normal, debajo.

CampoTexto.java nuevo

Caja de texto para nombres · 97 líneas · `Totito/src/CampoTexto.java`

`JTextField` redondeado con borde neón, texto de ayuda cuando está vacío y límite de 14 caracteres.

```
1  import java.awt.Color;
2  import java.awt.Font;
3  import java.awt.FontMetrics;
4  import java.awt.Graphics;
5  import java.awt.Graphics2D;
6  import java.awt.event.FocusAdapter;
7  import java.awt.event.FocusEvent;
8  import java.awt.geom.RoundRectangle2D;
9  import javax.swing.BorderFactory;
10 import javax.swing.JTextField;
11 import javax.swing.text.AbstractDocument;
12 import javax.swing.text.AttributeSet;
13 import javax.swing.text.BadLocationException;
14 import javax.swing.text.DocumentFilter;
15
```

Líneas 1-15 Imports, incluidos los de `javax.swing.text` para el filtro de caracteres.

```
16 /**
17  * Caja de texto redondeada con borde de neon, para escribir el nombre de
18  * cada jugador. Cuando esta vacia muestra un texto de ayuda en gris
19  * (placeholder) y no deja escribir mas de MAXIMO caracteres, para que el
20  * nombre quepa en el marcador y en la pantalla de victoria.
21  */
22 public class CampoTexto extends JTextField {
23
```

Líneas 16-23 Hereda de `JTextField`: escribir, seleccionar, copiar y pegar siguen funcionando.

```
24     public static final int MAXIMO = 14;
25     /** Espacio para el halo del borde. */
26     private static final int MARGEN = 5;
27
28     private final Color color;
29     private final String ayuda;
30
```

Líneas 24-30 Máximo de 14 letras, para que el nombre quepa en el marcador y en el título de victoria.

```
31     public CampoTexto(String ayuda, Color color) {
32         this.ayuda = ayuda;
33         this.color = color;
34         setOpaque(false);
35         setForeground(Tema.TEXTO);
36         setDisabledTextColor(Tema.TEXTO_SUAVE);
37         setCaretColor(Tema.TEXTO);
38         setSelectionColor(Tema.conAlfa(color, 120));
39         setSelectedTextColor(Tema.TEXTO);
40         setFont(Tema.fuente(Font.PLAIN, 17f));
41         setBorder(BorderFactory.createEmptyBorder(MARGEN + 4, MARGEN + 14, MARGEN + 4, MARGEN + 10));
```

Líneas 31-41 Transparente; colores del texto, del cursor y de la selección; fuente; y un borde vacío que deja el espacio interior donde va el texto.

```
42         ((AbstractDocument) getDocument()).setDocumentFilter(new LimiteDeLargo());
```

Línea 42 Instala el filtro que limita el largo (clase interna al final).

```
43          // Al entrar o salir del campo cambia el brillo del borde.
44          addFocusListener(new FocusAdapter() {
45              @Override
46              public void focusGained(FocusEvent e) {
47                  repaint();
48              }
49
50              @Override
51              public void focusLost(FocusEvent e) {
52                  repaint();
53              }
54          });
55      }
56
```

Líneas 43-56 Cuando el campo gana o pierde el foco se redibuja, para que el borde brille más mientras se escribe.

```
57      @Override
58      protected void paintComponent(Graphics g) {
59          Graphics2D g2 = Dibujo.preparar(g);
60          RoundRectangle2D forma = new RoundRectangle2D.Double(MARGEN, MARGEN,
61                  getWidth() - 2.0 * MARGEN, getHeight() - 2.0 * MARGEN, 18, 18);
62          g2.setColor(Tema.conAlfa(color, isEnabled() ? 26 : 12));
63          g2.fill(forma);
64          Dibujo.trazoNeon(g2, forma, color, 1.6f, hasFocus() ? 1.2f : 0.6f);
65
```

Líneas 57-65 Fondo redondeado y borde neón, más intenso si tiene el foco.

```
66          if (getText().isEmpty() && !hasFocus()) {
67              g2.setFont(getFont());
68              g2.setColor(Tema.TEXTO_SUAVE);
69              FontMetrics fm = g2.getFontMetrics();
70              g2.drawString(ayuda, getInsets().left, Dibujo.lineaBaseCentrada(fm, getHeight() / 2f));
71          }
72          g2.dispose();
```

Líneas 66-72 Texto de ayuda ("Jugador 1") en gris si el campo está vacío y no tiene el foco.

```
73          super.paintComponent(g);   // el texto y el cursor los pinta Swing
74      }
75
```

Líneas 73-75 Al final, `super.paintComponent` deja que Swing pinte el texto y el cursor encima de nuestro fondo. Como el campo es transparente, Swing no tapa lo que dibujamos.

```
76      /** Impide que el texto pase de MAXIMO caracteres. */
77      private static class LimiteDeLargo extends DocumentFilter {
78          @Override
79          public void insertString(FilterBypass fb, int offset, String texto, AttributeSet attr)
80                  throws BadLocationException {
81              replace(fb, offset, 0, texto, attr);
82          }
83
```

Líneas 76-83 Un `DocumentFilter` intercepta cada cambio del texto antes de que ocurra.

```
84          @Override
85          public void replace(FilterBypass fb, int offset, int largo, String texto, AttributeSet attr)
86                  throws BadLocationException {
87              if (texto == null) {
88                  texto = "";
89              }
90              int disponible = MAXIMO - (fb.getDocument().getLength() - largo);
91              if (texto.length() > disponible) {
92                  texto = texto.substring(0, Math.max(0, disponible));
93              }
94              super.replace(fb, offset, largo, texto, attr);
95          }
96      }
97  }
```

Líneas 84–97 Calcula cuántos caracteres caben todavía y recorta lo que se escribe o pega. `insertString` reutiliza este mismo método.

PanelTablero.java nuevo

Tablero 3×3 dibujado · 197 líneas · `Totito/src/PanelTablero.java`

Dibuja un `Tablero` con fichas neón y la raya ganadora, y avisa de los clics. No conoce las reglas.

```
1  import java.awt.AlphaComposite;
2  import java.awt.Color;
3  import java.awt.Cursor;
4  import java.awt.Graphics;
5  import java.awt.Graphics2D;
6  import java.awt.event.MouseAdapter;
7  import java.awt.event.MouseEvent;
8  import java.awt.geom.Line2D;
9  import java.awt.geom.RoundRectangle2D;
10 import javax.swing.JComponent;
11
```

Líneas 1-11 Imports.

```
12 /**
13  * Dibuja un Tablero de 3x3 con fichas de neon y, si alguien gano, la raya
14  * sobre las tres fichas de la linea ganadora.
15  *
16  * Solo DIBUJA: no sabe de reglas ni de turnos. Cuando el usuario hace clic
17  * en una casilla avisa al OyenteCasilla y es la pantalla quien decide que
18  * hacer. Se usa en tres sitios: la partida (interactivo), la pantalla de
19  * victoria y el ejemplo de las instrucciones (solo de muestra).
20  */
21 public class PanelTablero extends JComponent {
22
```

Líneas 12-22 Se usa en tres lugares: la partida (con clics), la pantalla de resultado y el ejemplo de las instrucciones (solo muestra).

```
23     /** Quien quiera enterarse de los clics implementa esta interfaz. */
24     public interface OyenteCasilla {
25         void casillaPulsada(int fila, int columna);
26     }
27
```

Líneas 23-27 El "oyente": quien quiera enterarse de los clics implementa este método. Es el mismo patrón que `ActionListener` en los botones.

```
28     private static final int MARGEN = 10;   // espacio para el halo del borde
29     private static final int RELLENO = 8;   // del borde a la primera casilla
30
```

Líneas 28-30 Medidas: margen para el halo y relleno hasta la primera casilla.

```
31     private final char[][] fichas = new char[3][3];
32     private int[] lineaGanadora;            // {fila1, col1, fila2, col2} o null
33     private OyenteCasilla oyente;
34     private boolean interactivo;
35     /** Ficha que se ve transparente al pasar el raton (la del jugador en turno). */
36     private char fichaFantasma = Tablero.VACIA;
37     private int filaEncima = -1;
38     private int columnaEncima = -1;
39
```

Líneas 31-39 Estado **para dibujar**: una copia de las fichas, la línea ganadora, el oyente, si acepta clics, la ficha "fantasma" y sobre qué casilla está el ratón (-1 = ninguna).

```
40      /** @param tamCasilla lado de cada casilla en pixeles. */
41      public PanelTablero(int tamCasilla) {
42          int lado = 2 * (MARGEN + RELLENO) + 3 * tamCasilla;
43          Dibujo.fijarTamano(this, lado, lado);
44          setAlignmentX(CENTER_ALIGNMENT);
45          for (char[] fila : fichas) {
46              java.util.Arrays.fill(fila, Tablero.VACIA);
47          }
48
```

Líneas 40-48 El tamaño total sale del tamaño de casilla: 3 casillas + márgenes. Las fichas empiezan vacías.

```
49          MouseAdapter raton = new MouseAdapter() {
50              @Override
51              public void mouseMoved(MouseEvent e) {
52                  actualizarCasillaEncima(e.getX(), e.getY());
53              }
54
55              @Override
56              public void mouseExited(MouseEvent e) {
57                  actualizarCasillaEncima(-1, -1);
58              }
59
```

Líneas 49-59 Escucha el ratón: al moverse actualiza la casilla resaltada y al salir la borra.

```
60              @Override
61              public void mousePressed(MouseEvent e) {
62                  actualizarCasillaEncima(e.getX(), e.getY());
63                  if (interactivo && oyente != null && filaEncima >= 0) {
64                      oyente.casillaPulsada(filaEncima, columnaEncima);
65                  }
66              }
67          };
```

Líneas 60-67 Al apretar: si se puede jugar, avisa al oyente con la fila y la columna. Se usa `mousePressed` y no `mouseClicked` porque este último falla si el ratón se mueve un poco durante el clic.

```
68          addMouseListener(raton);
69          addMouseMotionListener(raton);
70      }
71
```

Líneas 68-71 El mismo objeto atiende los clics y el movimiento.

```
72      public void setOyente(OyenteCasilla oyente) {
73          this.oyente = oyente;
74      }
75
76      public void setInteractivo(boolean interactivo) {
77          this.interactivo = interactivo;
78          actualizarCursor();
79          repaint();
80      }
81
82      public void setFichaFantasma(char ficha) {
83          this.fichaFantasma = ficha;
84          repaint();
85      }
86
```

Líneas 72-86 Setters: quién escucha, si es interactivo y qué ficha mostrar como fantasma. Cada uno pide un `repaint()`.

```
87      /** Copia el contenido del tablero para dibujarlo y busca la linea ganadora. */
88      public void mostrar(Tablero tablero) {
89          for (int fila = 0; fila < 3; fila++) {
90              for (int columna = 0; columna < 3; columna++) {
91                  fichas[fila][columna] = tablero.obtenerCasilla(fila, columna);
92              }
93          }
94          lineaGanadora = Partida.buscarLineaGanadora(tablero);
95          actualizarCursor();
96          repaint();
97      }
98
```

Líneas 87-98 Copia las 9 casillas con `obtenerCasilla` y busca la línea ganadora. El panel guarda su propia copia; así no depende de cuándo cambie el tablero real.

```
99      private double tamCasilla() {
100         return (getWidth() - 2.0 * (MARGEN + RELLENO)) / 3.0;
101     }
102
103     private double centroX(int columna) {
104         return MARGEN + RELLENO + (columna + 0.5) * tamCasilla();
105     }
106
107     private double centroY(int fila) {
108         return MARGEN + RELLENO + (fila + 0.5) * tamCasilla();
109     }
110
```

Líneas 99-110 Geometría: tamaño de cada casilla y centro de una columna o fila, en píxeles.

```
111     private void actualizarCasillaEncima(int x, int y) {
112         double tam = tamCasilla();
113         int columna = (int) Math.floor((x - MARGEN - RELLENO) / tam);
114         int fila = (int) Math.floor((y - MARGEN - RELLENO) / tam);
115         boolean dentro = x >= 0 && fila >= 0 && fila < 3 && columna >= 0 && columna < 3;
116         int nuevaFila = dentro ? fila : -1;
117         int nuevaColumna = dentro ? columna : -1;
118         if (nuevaFila != filaEncima || nuevaColumna != columnaEncima) {
119             filaEncima = nuevaFila;
120             columnaEncima = nuevaColumna;
121             actualizarCursor();
122             repaint();
123         }
124     }
125
```

Líneas 111-125 Convierte la posición del ratón en fila y columna dividiendo entre el tamaño de casilla. Solo redibuja si cambió de casilla, para no repintar en cada píxel.

```
126     /** Manita solo sobre casillas vacias y cuando se puede jugar. */
127     private void actualizarCursor() {
128         boolean sePuedeJugar = interactivo && filaEncima >= 0
129                 && fichas[filaEncima][columnaEncima] == Tablero.VACIA;
130         setCursor(Cursor.getPredefinedCursor(sePuedeJugar ? Cursor.HAND_CURSOR : Cursor.DEFAULT_CURSOR));
131     }
132
```

Líneas 126-132 Pone la mano solo sobre casillas vacías cuando se puede jugar; así el usuario sabe dónde puede hacer clic.

```
133     @Override
134     protected void paintComponent(Graphics g) {
135         Graphics2D g2 = Dibujo.preparar(g);
136         double tam = tamCasilla();
137         double inicio = MARGEN + RELLENO;
138 
139         // Marco exterior.
140         RoundRectangle2D marco = new RoundRectangle2D.Double(MARGEN, MARGEN,
141                 getWidth() - 2.0 * MARGEN, getHeight() - 2.0 * MARGEN, 22, 22);
142         g2.setColor(Tema.conAlfa(Tema.FONDO_ABAJO, 150));
143         g2.fill(marco);
144         Dibujo.trazoNeon(g2, marco, Tema.MORADO, 1.5f, 0.8f);
145 
```

Líneas 133-145 Dibujo, por capas (lo de abajo primero). Primera capa: el marco exterior con borde neón.

```
146         // Casillas.
147         for (int fila = 0; fila < 3; fila++) {
148             for (int columna = 0; columna < 3; columna++) {
149                 RoundRectangle2D casilla = new RoundRectangle2D.Double(
150                         inicio + columna * tam + 4, inicio + fila * tam + 4, tam - 8, tam - 8, 14, 14);
151                 boolean resaltar = interactivo && fila == filaEncima && columna == columnaEncima
152                         && fichas[fila][columna] == Tablero.VACIA;
153                 g2.setColor(resaltar ? Tema.conAlfa(Tema.LAVANDA, 40) : Tema.CASILLA);
154                 g2.fill(casilla);
155             }
156         }
157 
```

Líneas 146-157 Las 9 casillas redondeadas. La que está bajo el ratón se aclara si se puede jugar.

```
158         // Rejilla: las dos lineas verticales y las dos horizontales.
159         Color colorRejilla = Tema.conAlfa(Tema.LAVANDA, 150);
160         for (int i = 1; i < 3; i++) {
161             double p = inicio + i * tam;
162             Dibujo.trazoNeon(g2, new Line2D.Double(p, MARGEN + 2, p, getHeight() - MARGEN - 2), colorRejilla, 1.
    5f, 0.4f);
163             Dibujo.trazoNeon(g2, new Line2D.Double(MARGEN + 2, p, getWidth() - MARGEN - 2, p), colorRejilla, 1.5
    f, 0.4f);
164         }
165 
```

Líneas 158-165 Las 4 líneas de la rejilla, con un neón tenue.

```
166         // Fichas.
167         float grosor = (float) Math.max(3, tam * 0.085);
168         for (int fila = 0; fila < 3; fila++) {
169             for (int columna = 0; columna < 3; columna++) {
170                 Dibujo.dibujarFicha(g2, fichas[fila][columna], centroX(columna), centroY(fila), tam * 0.56, gros
    or);
171             }
172         }
173 
```

Líneas 166-173 Cada ficha en el centro de su casilla. El grosor del trazo es proporcional al tamaño de casilla, así se ve igual en el tablero grande y en el chico.

```
174         // Ficha "fantasma" bajo el raton: muestra donde caeria la jugada.
175         if (interactivo && filaEncima >= 0 && fichas[filaEncima][columnaEncima] == Tablero.VACIA
176                 && fichaFantasma != Tablero.VACIA) {
177             g2.setComposite(AlphaComposite.getInstance(AlphaComposite.SRC_OVER, 0.28f));
178             Dibujo.dibujarFicha(g2, fichaFantasma, centroX(columnaEncima), centroY(filaEncima), tam * 0.56, gros
    or);
179             g2.setComposite(AlphaComposite.SrcOver);
180         }
181
```

Líneas 174-181 Vista previa: la ficha de quien tiene el turno, al 28 % de opacidad, en la casilla bajo el ratón.

```
182         // Raya sobre la linea ganadora, un poco mas larga que las fichas.
183         if (lineaGanadora != null) {
184             double x1 = centroX(lineaGanadora[1]);
185             double y1 = centroY(lineaGanadora[0]);
186             double x2 = centroX(lineaGanadora[3]);
187             double y2 = centroY(lineaGanadora[2]);
188             double largo = Math.hypot(x2 - x1, y2 - y1);
189             double dx = (x2 - x1) / largo * tam * 0.38;
190             double dy = (y2 - y1) / largo * tam * 0.38;
191             Color color = Tema.colorDeFicha(fichas[lineaGanadora[0]][lineaGanadora[1]]);
192             Dibujo.trazoNeon(g2, new Line2D.Double(x1 - dx, y1 - dy, x2 + dx, y2 + dy), color,
193                     (float) Math.max(3, tam * 0.06), 1.4f);
194         }
195         g2.dispose();
196     }
197 }
```

Líneas 182-197 La raya ganadora va del centro de la primera casilla al de la última, alargada un poco por los dos lados (se normaliza la dirección dividiendo entre el largo). Usa el color del ganador y brilla más (intensidad 1.4).

TarjetaJugador.java nuevo

Tarjeta de nombre de jugador · 85 líneas · `Totito/src/TarjetaJugador.java`

La tarjeta de la configuración con icono, título, subrayado y la caja para el nombre.

```
1  import java.awt.Color;
2  import java.awt.Font;
3  import java.awt.GradientPaint;
4  import java.awt.Graphics;
5  import java.awt.Graphics2D;
6  import java.awt.geom.Line2D;
7
```

Líneas 1-7 Imports.

```
8  /**
9   * Tarjeta de la pantalla de configuracion donde se escribe el nombre de un
10  * jugador. En el modo "Contra la IA" la tarjeta del Jugador 2 se bloquea y
11  * pasa a llamarse "Computadora" (ver setComputadora).
12  */
13 public class TarjetaJugador extends Tarjeta {
14
```

Líneas 8-14 Hereda de `Tarjeta`: reutiliza la caja de cristal y le agrega su contenido.

```
15     public static final int ANCHO = 290;
16     public static final int ALTO = 150;
17     public static final String NOMBRE_IA = "Computadora";
18
```

Líneas 15-18 Tamaño fijo y el nombre que se pone cuando el Jugador 2 es la computadora.

```
19     private final String titulo;
20     private final Color color;
21     private final CampoTexto campo;
22     private boolean esComputadora;
23     /** Lo que el usuario habia escrito antes de pasar a modo IA, para devolverselo. */
24     private String nombreGuardado = "";
25
```

Líneas 19-25 Estado: título, color, la caja de texto, si es la computadora y el nombre que había escrito el usuario antes de pasar a modo IA.

```
26     public TarjetaJugador(String titulo, Color color) {
27         super(color, 26);
28         this.titulo = titulo;
29         this.color = color;
30         setLayout(null);   // el campo se coloca a mano en doLayout()
31         campo = new CampoTexto(titulo, color);
32         add(campo);
33         Dibujo.fijarTamano(this, ANCHO, ALTO);
34     }
35
```

Líneas 26-35 Sin layout (`null`): el campo se coloca a mano. El texto de ayuda del campo es el propio título.

```
36      @Override
37      public void doLayout() {
38          campo.setBounds(MARGEN + 12, getHeight() - MARGEN - 10 - 46, getWidth() - 2 * MARGEN - 24, 46);
39      }
40
```

Líneas 36-40 Pone el campo abajo, ocupando el ancho de la tarjeta menos los márgenes.

```
41      /** El nombre escrito o, si se dejo vacio, el titulo ("Jugador 1"). */
42      public String getNombre() {
43          String nombre = campo.getText().trim();
44          return nombre.isEmpty() ? titulo : nombre;
45      }
46
```

Líneas 41-46 Devuelve el nombre sin espacios sobrantes (`trim`). Si se dejó vacío, devuelve el título; así nunca hay un jugador sin nombre.

```
47      /** Convierte la tarjeta en la de la computadora (true) o la devuelve a humano (false). */
48      public void setComputadora(boolean computadora) {
49          if (computadora == esComputadora) {
50              return;
51          }
52          esComputadora = computadora;
53          if (computadora) {
54              nombreGuardado = campo.getText();
55              campo.setText(NOMBRE_IA);
56          } else {
57              campo.setText(nombreGuardado);
58          }
59          campo.setEnabled(!computadora);
60          repaint();
61      }
62
```

Líneas 47-62 Al pasar a modo IA guarda lo escrito, pone "Computadora" y bloquea el campo. Al volver, devuelve el nombre guardado y lo desbloquea; así no se pierde lo que se había escrito.

```
63      @Override
64      protected void paintComponent(Graphics g) {
65          super.paintComponent(g);   // la caja de cristal
66          Graphics2D g2 = Dibujo.preparar(g);
67
68          Iconos.Icono icono = esComputadora ? Iconos.COMPUTADORA : Iconos.PERSONA;
69          icono.dibujar(g2, MARGEN + 22, MARGEN + 10, 42, color);
70
71          g2.setFont(Tema.fuente(Font.BOLD, 21f));
72          g2.setColor(Tema.TEXTO);
73          float xTitulo = MARGEN + 88;
74          float base = MARGEN + 36;
75          g2.drawString(titulo, xTitulo, base);
76          g2.setPaint(new GradientPaint(xTitulo, 0, color, xTitulo + 140, 0, Tema.conAlfa(color, 0)));
77          g2.setStroke(Dibujo.redondo(3f));
78          g2.draw(new Line2D.Float(xTitulo, base + 12, xTitulo + 140, base + 12));
79
80          g2.setFont(Tema.fuente(Font.PLAIN, 13f));
81          g2.setColor(Tema.TEXTO);
82          g2.drawString("Nombre", MARGEN + 18, MARGEN + 70);
83          g2.dispose();
84      }
85  }
```

Líneas 63-85 Primero `super.paintComponent` (la caja de cristal) y encima el icono (persona o chip), el título, su subrayado degradado y la etiqueta "Nombre".

SelectorSimbolo.java nuevo

Elegir X u O · 107 líneas · `Totito/src/SelectorSimbolo.java`

Las dos cajas X / O. La marcada es la del Jugador 1; el Jugador 2 se queda con la otra.

```
1  import java.awt.AlphaComposite;
2  import java.awt.Color;
3  import java.awt.Cursor;
4  import java.awt.Font;
5  import java.awt.FontMetrics;
6  import java.awt.Graphics;
7  import java.awt.Graphics2D;
8  import java.awt.Rectangle;
9  import java.awt.event.MouseAdapter;
10 import java.awt.event.MouseEvent;
11 import java.awt.geom.Ellipse2D;
12 import java.awt.geom.RoundRectangle2D;
13 import javax.swing.JComponent;
14
```

Líneas 1-14 Imports.

```
15 /**
16  * Las dos cajas X / O de la pantalla de configuracion. La caja marcada es
17  * el simbolo del Jugador 1; el Jugador 2 se queda con el otro. Debajo se
18  * escribe una leyenda para que no haya dudas de quien juega con que.
19  */
20 public class SelectorSimbolo extends JComponent {
21
```

Líneas 15-21 El comentario explica la regla de la selección.

```
22     private static final int ANCHO_CAJA = 120;
23     private static final int ALTO_CAJA = 98;
24     private static final int HUECO = 34;
25     private static final int ALTO_LEYENDA = 22;
26
```

Líneas 22-26 Medidas de las cajas y de la leyenda.

```
27     private char fichaJugador1 = 'X';
28
```

Líneas 27-28 El único estado: qué ficha eligió el Jugador 1 (empieza en X).

```
29     public SelectorSimbolo() {
30         Dibujo.fijarTamano(this, 2 * ANCHO_CAJA + HUECO + 160, ALTO_CAJA + ALTO_LEYENDA);
31         setAlignmentX(CENTER_ALIGNMENT);
32         setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
```

Líneas 29-32 Tamaño fijo, centrado y mano como cursor.

```
33          addMouseListener(new MouseAdapter() {
34              @Override
35              public void mousePressed(MouseEvent e) {
36                  if (caja('X').contains(e.getPoint())) {
37                      setFichaJugador1('X');
38                  } else if (caja('O').contains(e.getPoint())) {
39                      setFichaJugador1('O');
40                  }
41              }
42          });
43      }
44
```

Líneas 33-44 Al hacer clic se mira en qué caja cayó el punto (`Rectangle.contains`) y se elige esa ficha.

```
45      public char getFichaJugador1() {
46          return fichaJugador1;
47      }
48
49      public void setFichaJugador1(char ficha) {
50          fichaJugador1 = ficha;
51          repaint();
52      }
53
```

Líneas 45-53 Lectura y cambio de la elección. Cambiarla redibuja el componente.

```
54      /** Zona (en pixeles) de la caja de cada ficha. */
55      private Rectangle caja(char ficha) {
56          int x = (getWidth() - 2 * ANCHO_CAJA - HUECO) / 2;
57          if (ficha == 'O') {
58              x += ANCHO_CAJA + HUECO;
59          }
60          return new Rectangle(x, 0, ANCHO_CAJA, ALTO_CAJA);
61      }
62
```

Líneas 54-62 Calcula la posición de cada caja: las dos centradas, separadas por `HUECO` .

```
63      @Override
64      protected void paintComponent(Graphics g) {
65          Graphics2D g2 = Dibujo.preparar(g);
66          for (char ficha : new char[]{'X', 'O'}) {
67              pintarCaja(g2, ficha, ficha == fichaJugador1);
68          }
69
```

Líneas 63-69 Dibuja las dos cajas...

```
70          char fichaJugador2 = fichaJugador1 == 'X' ? 'O' : 'X';
71          String leyenda = "Jugador 1 juega con " + fichaJugador1 + "   ·   Jugador 2 con " + fichaJugador2;
72          g2.setFont(Tema.fuente(Font.PLAIN, 13f));
73          FontMetrics fm = g2.getFontMetrics();
74          g2.setColor(Tema.TEXTO_SUAVE);
75          g2.drawString(leyenda, (getWidth() - fm.stringWidth(leyenda)) / 2f, getHeight() - 5);
76          g2.dispose();
77      }
78
```

Líneas 70-78 ...y la leyenda de abajo ("Jugador 1 juega con X · Jugador 2 con O"), que cambia según la selección.

```
79      private void pintarCaja(Graphics2D g2, char ficha, boolean elegida) {
80          Rectangle r = caja(ficha);
81          Color color = Tema.colorDeFicha(ficha);
82          RoundRectangle2D forma = new RoundRectangle2D.Double(r.x + 8, r.y + 8, r.width - 16, r.height - 16, 20,
    20);
83          g2.setColor(Tema.conAlfa(color, elegida ? 40 : 14));
84          g2.fill(forma);
85          Dibujo.trazoNeon(g2, forma, color, 2f, elegida ? 1.3f : 0.4f);
86
```

Líneas 79-86 Cada caja: relleno y borde del color de la ficha. La elegida tiene más relleno y brillo.

```
87          // La ficha no elegida se ve apagada.
88          if (!elegida) {
89              g2.setComposite(AlphaComposite.getInstance(AlphaComposite.SRC_OVER, 0.5f));
90          }
91          double cx = r.getCenterX();
92          double cy = r.y + 8 + (r.height - 16) * 0.4;
93          Dibujo.dibujarFicha(g2, ficha, cx, cy, (r.height - 16) * 0.44, 6.5f);
94          g2.setComposite(AlphaComposite.SrcOver);
95
```

Líneas 87-95 La ficha no elegida se dibuja al 50 % de opacidad.

```
96          // Boton de radio debajo de la ficha.
97          double cyRadio = r.y + r.height - 8 - 15;
98          Ellipse2D aro = new Ellipse2D.Double(cx - 8, cyRadio - 8, 16, 16);
99          if (elegida) {
100             Dibujo.trazoNeon(g2, aro, color, 2f, 0.8f);
101             g2.setColor(Tema.aclarar(color, 0.5f));
102             g2.fill(new Ellipse2D.Double(cx - 4.5, cyRadio - 4.5, 9, 9));
103         } else {
104             Dibujo.trazo(g2, aro, Tema.conAlfa(Tema.LAVANDA, 120), 1.8f);
105         }
106     }
107 }
```

Líneas 96-107 El "botón de radio" de abajo: en la elegida, un aro neón con un punto adentro; en la otra, solo un aro gris.

OpcionModo.java nuevo

Opción de modo de juego · 85 líneas · `Totito/src/OpcionModo.java`

Tarjeta tipo botón de radio: "2 jugadores" o "Contra la IA".

```
1  import java.awt.Cursor;
2  import java.awt.Font;
3  import java.awt.Graphics;
4  import java.awt.Graphics2D;
5  import java.awt.event.MouseAdapter;
6  import java.awt.event.MouseEvent;
7  import java.awt.geom.Ellipse2D;
8  import java.awt.geom.RoundRectangle2D;
9  import javax.swing.JComponent;
10
```

Líneas 1-10 Imports.

```
11 /**
12  * Una opcion del "Modo de juego" (boton de radio con icono, titulo y
13  * descripcion). La pantalla que la usa decide cual queda marcada: al hacer
14  * clic solo se avisa con el Runnable de setAlSeleccionar().
15  */
16 public class OpcionModo extends JComponent {
17
```

Líneas 11-17 La opción no decide sola si queda marcada: avisa con un `Runnable` y la pantalla marca una y desmarca la otra. Así las dos opciones se comportan como un grupo.

```
18     public static final int ANCHO = 290;
19     public static final int ALTO = 72;
20     private static final int MARGEN = 7;
21
```

Líneas 18-21 Tamaño fijo.

```
22     private final Iconos.Icono icono;
23     private final String titulo;
24     private final String descripcion;
25     private boolean seleccionada;
26     private Runnable alSeleccionar;
27
```

Líneas 22-27 Datos: icono, título, descripción, si está seleccionada y qué hacer al pulsarla.

```
28     public OpcionModo(Iconos.Icono icono, String titulo, String descripcion) {
29         this.icono = icono;
30         this.titulo = titulo;
31         this.descripcion = descripcion;
32         Dibujo.fijarTamano(this, ANCHO, ALTO);
33         setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
34         addMouseListener(new MouseAdapter() {
35             @Override
36             public void mousePressed(MouseEvent e) {
37                 if (alSeleccionar != null) {
38                     alSeleccionar.run();
39                 }
40             }
41         });
42     }
43
```

Líneas 28-43 Constructor: tamaño, mano como cursor y, al apretar, se ejecuta la acción.

```
44      public void setAlSeleccionar(Runnable accion) {
45          this.alSeleccionar = accion;
46      }
47
```

Líneas 44-47 Configura la acción al pulsar.

```
48      public void setSeleccionada(boolean seleccionada) {
49          this.seleccionada = seleccionada;
50          repaint();
51      }
52
```

Líneas 48-52 Marca o desmarca la opción y redibuja.

```
53      @Override
54      protected void paintComponent(Graphics g) {
55          Graphics2D g2 = Dibujo.preparar(g);
56          RoundRectangle2D forma = new RoundRectangle2D.Double(MARGEN, MARGEN,
57                  getWidth() - 2.0 * MARGEN, getHeight() - 2.0 * MARGEN, 22, 22);
58          g2.setColor(Tema.conAlfa(Tema.MORADO, seleccionada ? 38 : 12));
59          g2.fill(forma);
60          Dibujo.trazoNeon(g2, forma, seleccionada ? Tema.aclarar(Tema.MORADO, 0.15f) : Tema.MORADO, 1.8f,
61                  seleccionada ? 1.2f : 0.35f);
62
```

Líneas 53-62 Caja con relleno y borde morado, más intensos si está seleccionada.

```
63          // Boton de radio.
64          double cx = MARGEN + 26;
65          double cy = getHeight() / 2.0;
66          Ellipse2D aro = new Ellipse2D.Double(cx - 10, cy - 10, 20, 20);
67          if (seleccionada) {
68              Dibujo.trazoNeon(g2, aro, Tema.ROSA, 2.2f, 0.9f);
69              g2.setColor(Tema.aclarar(Tema.ROSA, 0.45f));
70              g2.fill(new Ellipse2D.Double(cx - 5, cy - 5, 10, 10));
71          } else {
72              Dibujo.trazo(g2, aro, Tema.conAlfa(Tema.LAVANDA, 110), 1.8f);
73          }
74
```

Líneas 63-74 Aro a la izquierda: rosa con un punto si está seleccionada, gris si no.

```
75          icono.dibujar(g2, MARGEN + 48, cy - 17, 34, seleccionada ? Tema.LAVANDA : Tema.TEXTO_SUAVE);
76
```

Líneas 75-76 Icono (personas o chip).

```
77          g2.setFont(Tema.fuente(Font.BOLD, 16f));
78          g2.setColor(seleccionada ? Tema.TEXTO : Tema.TEXTO_SUAVE);
79          g2.drawString(titulo, MARGEN + 92, (float) cy - 3);
80          g2.setFont(Tema.fuente(Font.PLAIN, 12f));
81          g2.setColor(Tema.TEXTO_SUAVE);
82          g2.drawString(descripcion, MARGEN + 92, (float) cy + 15);
83          g2.dispose();
84      }
85  }
```

Líneas 77-85 Título en negrita y descripción en letra pequeña. Si no está seleccionada, todo se ve más apagado.

PanelMarcador.java nuevo

Marcador de la partida · 80 líneas · `Totito/src/PanelMarcador.java`

Una ficha por jugador con su símbolo, nombre y victorias, y los empates en el centro.

```
1  import java.awt.Color;
2  import java.awt.Font;
3  import java.awt.FontMetrics;
4  import java.awt.Graphics;
5  import java.awt.Graphics2D;
6  import java.awt.geom.RoundRectangle2D;
7  import javax.swing.JComponent;
8
```

Líneas 1-8 Imports.

```
9  /**
10  * Marcador de la pantalla de juego: una "ficha" por jugador con su
11  * simbolo, nombre y victorias, y los empates en el centro. La ficha del
12  * jugador que tiene el turno brilla mas.
13  */
14 public class PanelMarcador extends JComponent {
15
```

Líneas 9-15 El comentario lo describe.

```
16     private static final int ANCHO = 590;
17     private static final int ALTO = 82;
18     private static final int ANCHO_CENTRO = 104;
19
20     private Partida partida;
21
```

Líneas 16-21 Medidas: ancho total, alto y el espacio del centro para los empates.

```
22     public PanelMarcador() {
23         Dibujo.fijarTamano(this, ANCHO, ALTO);
24         setAlignmentX(CENTER_ALIGNMENT);
25     }
26
```

Líneas 22-26 Tamaño fijo.

```
27     public void mostrar(Partida partida) {
28         this.partida = partida;
29         repaint();
30     }
31
```

Líneas 27-31 Recibe la partida y se redibuja.

```
32      @Override
33      protected void paintComponent(Graphics g) {
34          if (partida == null) {
35              return;
36          }
37          Graphics2D g2 = Dibujo.preparar(g);
38          int anchoFicha = (getWidth() - ANCHO_CENTRO) / 2;
39          boolean enCurso = partida.getEstado() == Partida.Estado.EN_CURSO;
40          for (int i = 0; i < 2; i++) {
41              Jugador jugador = partida.getJugador(i);
42              boolean tieneTurno = enCurso && partida.getJugadorEnTurno() == jugador;
43              int x = i == 0 ? 0 : getWidth() - anchoFicha;
44              pintarJugador(g2, x, anchoFicha, jugador, partida.getVictorias(i), tieneTurno);
45          }
46
```

Líneas 32-46 Si todavía no hay partida, no dibuja nada. Si hay, dibuja la ficha de cada jugador: la de la izquierda para el Jugador 1 y la de la derecha para el 2. Brilla la del jugador que tiene el turno.

```
47          // Empates en el centro.
48          float centro = getWidth() / 2f;
49          g2.setFont(Tema.fuenteEspaciada(Font.BOLD, 11f, 0.2f));
50          FontMetrics fm = g2.getFontMetrics();
51          g2.setColor(Tema.TEXTO_SUAVE);
52          g2.drawString("EMPATES", centro - fm.stringWidth("EMPATES") / 2f, 32);
53          g2.setFont(Tema.fuente(Font.BOLD, 24f));
54          fm = g2.getFontMetrics();
55          String empates = String.valueOf(partida.getEmpates());
56          g2.setColor(Tema.LAVANDA);
57          g2.drawString(empates, centro - fm.stringWidth(empates) / 2f, 62);
58          g2.dispose();
59      }
60
```

Líneas 47-60 "EMPATES" en letras espaciadas y el número debajo, centrados.

```
61      private void pintarJugador(Graphics2D g2, int x, int ancho, Jugador jugador, int victorias, boolean tieneTur
    no) {
62          Color color = Tema.colorDeFicha(jugador.getFicha());
63          RoundRectangle2D forma = new RoundRectangle2D.Double(x + 8, 8, ancho - 16, getHeight() - 16, 22, 22);
64          g2.setColor(Tema.conAlfa(color, tieneTurno ? 45 : 14));
65          g2.fill(forma);
66          Dibujo.trazoNeon(g2, forma, color, 1.8f, tieneTurno ? 1.3f : 0.35f);
67
68          double cy = getHeight() / 2.0;
69          Dibujo.dibujarFicha(g2, jugador.getFicha(), x + 8 + 34, cy, 28, 4.5f);
70
71          int xTexto = x + 8 + 64;
72          g2.setFont(Tema.fuente(Font.BOLD, 17f));
73          FontMetrics fm = g2.getFontMetrics();
74          g2.setColor(Tema.TEXTO);
75          g2.drawString(Dibujo.recortar(fm, jugador.getNombre(), ancho - 16 - 64 - 12), xTexto, (float) cy - 2);
76          g2.setFont(Tema.fuente(Font.PLAIN, 13f));
77          g2.setColor(Tema.TEXTO_SUAVE);
78          g2.drawString("Victorias: " + victorias, xTexto, (float) cy + 17);
79      }
80  }
```

Líneas 61-80 Caja del color de su ficha, la ficha pequeña, el nombre (recortado con "…" si no cabe) y "Victorias: n".

PasoInstruccion.java nuevo

Paso numerado de las instrucciones · 50 líneas · `Totito/src/PasoInstruccion.java`

Un círculo neón con el número y el texto a la derecha.

```
1  import java.awt.Color;
2  import java.awt.Font;
3  import java.awt.FontMetrics;
4  import java.awt.Graphics;
5  import java.awt.Graphics2D;
6  import java.awt.geom.Ellipse2D;
7  import javax.swing.JLabel;
8  import javax.swing.JPanel;
9
```

Líneas 1-9 Imports.

```
10 /**
11  * Un paso numerado de "Cómo jugar": circulo de neon con el numero y el
12  * texto a la derecha. El texto es HTML de JLabel, asi se pueden pintar de
13  * color la X y la O dentro de la frase y partir la linea con &lt;br&gt;.
14  */
15 public class PasoInstruccion extends JPanel {
16
```

Líneas 10-16 El texto es HTML dentro de un `JLabel`: así se pueden colorear la X y la O dentro de la frase y partir la línea con `<br>`.

```
17     private static final int ALTO = 54;
18     private static final int DIAMETRO = 38;
19
20     private final int numero;
21     private final Color color;
22
```

Líneas 17-22 Alto del paso y diámetro del círculo.

```
23     public PasoInstruccion(int numero, Color color, String html, int ancho) {
24         super(null);
25         this.numero = numero;
26         this.color = color;
27         setOpaque(false);
28         setAlignmentX(CENTER_ALIGNMENT);
29         Dibujo.fijarTamano(this, ancho, ALTO);
30
```

Líneas 23-30 Panel transparente sin layout, de tamaño fijo.

```
31         JLabel texto = new JLabel("<html>" + html + "</html>");
32         texto.setFont(Tema.fuente(Font.PLAIN, 17f));
33         texto.setForeground(Tema.TEXTO);
34         texto.setBounds(DIAMETRO + 30, 0, ancho - DIAMETRO - 30, ALTO);
35         add(texto);
36     }
37
```

Líneas 31-37 La etiqueta HTML va a la derecha del círculo. Swing interpreta el texto como HTML si empieza con `<html>`.

```
38      @Override
39      protected void paintComponent(Graphics g) {
40          Graphics2D g2 = Dibujo.preparar(g);
41          double y = (getHeight() - DIAMETRO) / 2.0;
42          Dibujo.trazoNeon(g2, new Ellipse2D.Double(8, y, DIAMETRO, DIAMETRO), color, 2f, 0.9f);
43          g2.setFont(Tema.fuente(Font.BOLD, 18f));
44          FontMetrics fm = g2.getFontMetrics();
45          String n = String.valueOf(numero);
46          g2.setColor(Tema.TEXTO);
47          g2.drawString(n, 8 + (DIAMETRO - fm.stringWidth(n)) / 2f, Dibujo.lineaBaseCentrada(fm, getHeight() / 2
    f));
48          g2.dispose();
49      }
50  }
```

Líneas 38–50 El círculo neón y el número centrado adentro.

IconoResultado.java nuevo

Ilustración del resultado · 51 líneas · `Totito/src/IconoResultado.java`

Trofeo con destellos si alguien ganó; una X y una O juntas si hubo empate.

```
1  import java.awt.Color;
2  import java.awt.Graphics;
3  import java.awt.Graphics2D;
4  import java.awt.geom.Line2D;
5  import javax.swing.JComponent;
6
```

Líneas 1-6 Imports.

```
7  /**
8   * La ilustracion de arriba de la pantalla de resultado: un trofeo con
9   * destellos si alguien gano, o una X y una O juntas si hubo empate.
10  */
11 public class IconoResultado extends JComponent {
12
```

Líneas 7-12 El comentario lo describe.

```
13     /** Angulos (en grados) de los destellos alrededor del trofeo. */
14     private static final int[] ANGULOS = {145, 180, 215, 35, 0, -35};
15     private static final Color[] COLORES = {Tema.ROSA, Tema.CIAN, Tema.MORADO};
16
17     private boolean empate;
18
```

Líneas 13-18 Ángulos de los 6 destellos (3 a cada lado) y los colores que se van alternando.

```
19     public IconoResultado() {
20         Dibujo.fijarTamano(this, 260, 100);
21         setAlignmentX(CENTER_ALIGNMENT);
22     }
23
```

Líneas 19-23 Tamaño fijo.

```
24     public void setEmpate(boolean empate) {
25         this.empate = empate;
26         repaint();
27     }
28
```

Líneas 24-28 Cambia entre trofeo y empate.

```
29     @Override
30     protected void paintComponent(Graphics g) {
31         Graphics2D g2 = Dibujo.preparar(g);
32         double cx = getWidth() / 2.0;
33         double cy = getHeight() / 2.0;
```

Líneas 29-33 Calcula el centro del componente.

```
34         if (empate) {
35             Dibujo.dibujarFicha(g2, 'X', cx - 40, cy, 54, 7f);
36             Dibujo.dibujarFicha(g2, 'O', cx + 40, cy, 54, 7f);
37         } else {
```

Líneas 34-37 Empate: una X a la izquierda y una O a la derecha.

```
38              for (int i = 0; i < ANGULOS.length; i++) {
39                  double a = Math.toRadians(ANGULOS[i]);
40                  double r1 = 58;
41                  double r2 = i % 3 == 1 ? 80 : 72;   // el destello del medio es mas largo
42                  Line2D destello = new Line2D.Double(
43                          cx + Math.cos(a) * r1, cy - Math.sin(a) * r1,
44                          cx + Math.cos(a) * r2, cy - Math.sin(a) * r2);
45                  Dibujo.trazoNeon(g2, destello, COLORES[i % COLORES.length], 3.5f, 0.8f);
46              }
```

Líneas 38-46 Destellos: cada uno es una línea corta en dirección radial, de radio 58 a 72 (80 el del medio), calculada con coseno y seno del ángulo. Se resta el seno porque en pantalla la Y crece hacia abajo.

```
47              Iconos.TROFEO.dibujar(g2, cx - 42, cy - 44, 84, Tema.CIAN);
48          }
49          g2.dispose();
50      }
51  }
```

Líneas 47-51 El trofeo cian en el centro.

Parte F — Las pantallas

Cada pantalla hereda de `PantallaBase` y en su constructor apila componentes de arriba abajo. Las cuatro primeras corresponden a las imágenes del diseño; la pantalla de juego se hizo con el mismo estilo.

PantallaBase.java nuevo

Base de todas las pantallas · 116 líneas · `Totito/src/PantallaBase.java`

Pinta el fondo, pone el encabezado y ofrece métodos cómodos para apilar componentes.

```
1  import java.awt.BorderLayout;
2  import java.awt.Color;
3  import java.awt.Dimension;
4  import java.awt.GradientPaint;
5  import java.awt.Graphics;
6  import java.awt.Graphics2D;
7  import java.awt.RadialGradientPaint;
8  import java.awt.geom.Ellipse2D;
9  import java.awt.geom.Path2D;
10 import java.awt.geom.Point2D;
11 import javax.swing.BorderFactory;
12 import javax.swing.Box;
13 import javax.swing.BoxLayout;
14 import javax.swing.JComponent;
15 import javax.swing.JPanel;
16
```

Líneas 1-16 Imports.

```
17 /**
18  * Base de todas las pantallas: pinta el fondo (degradado, X y O gigantes,
19  * puntitos), pone el Encabezado arriba y deja un panel "contenido" en
20  * columna (BoxLayout vertical) donde cada pantalla agrega lo suyo.
21  *
22  * Para crear una pantalla nueva: extender esta clase y, en el constructor,
23  * usar agregar(), espacio() y relleno() en el orden en que se quieren ver
24  * las cosas de arriba abajo.
25  */
26 public abstract class PantallaBase extends JPanel {
27
```

Líneas 17-27 `abstract`: no se usa sola, solo como base. El comentario explica cómo crear una pantalla nueva.

```
28     private final JPanel contenido;
29
```

Líneas 28-29 El panel central donde cada pantalla pone sus cosas.

```
30      protected PantallaBase(Navegacion navegacion) {
31          super(new BorderLayout());
32          add(new Encabezado(navegacion::salir), BorderLayout.NORTH);
33
34          contenido = new JPanel();
35          contenido.setOpaque(false);
36          contenido.setLayout(new BoxLayout(contenido, BoxLayout.Y_AXIS));
37          contenido.setBorder(BorderFactory.createEmptyBorder(0, 0, 10, 0));
38          add(contenido, BorderLayout.CENTER);
39      }
40
```

Líneas 30-40 `BorderLayout`: el encabezado arriba (NORTH) y el contenido al centro (CENTER). El contenido usa `BoxLayout` vertical, que apila componentes en columna, y es transparente para que se vea el fondo.

```
41      /** Agrega un componente centrado debajo del anterior. */
42      protected void agregar(JComponent componente) {
43          componente.setAlignmentX(CENTER_ALIGNMENT);
44          contenido.add(componente);
45      }
46
```

Líneas 41-46 Agrega un componente centrado debajo del anterior.

```
47      /** Separacion vertical fija, en pixeles. */
48      protected void espacio(int alto) {
49          contenido.add(Box.createRigidArea(new Dimension(0, alto)));
50      }
51
```

Líneas 47-51 Separación fija. Se usa `createRigidArea` (de ancho 0) y no un *strut* porque el strut tiene ancho máximo infinito y haría que el contenedor se estirara.

```
52      /** Espacio elastico: poner uno al principio y otro al final centra todo en vertical. */
53      protected void relleno() {
54          contenido.add(Box.createVerticalGlue());
55      }
56
```

Líneas 52-56 Espacio elástico (*glue*): uno al principio y otro al final reparten el espacio sobrante y la columna queda centrada en vertical.

```
57      /** Pone varios componentes uno al lado del otro, separados por "hueco" pixeles. */
58      protected static JPanel fila(int hueco, JComponent... componentes) {
59          JPanel fila = new JPanel();
60          fila.setOpaque(false);
61          fila.setLayout(new BoxLayout(fila, BoxLayout.X_AXIS));
62          for (int i = 0; i < componentes.length; i++) {
63              if (i > 0) {
64                  fila.add(Box.createRigidArea(new Dimension(hueco, 0)));
65              }
66              fila.add(componentes[i]);
67          }
68          fila.setMaximumSize(fila.getPreferredSize());
69          return fila;
70      }
71
```

Líneas 57-71 Pone componentes uno al lado del otro (`BoxLayout` horizontal) con un hueco entre ellos. Se le fija el tamaño máximo para que no se estire.

```
72      @Override
73      protected void paintComponent(Graphics g) {
74          Graphics2D g2 = Dibujo.preparar(g);
75          int ancho = getWidth();
76          int alto = getHeight();
77
78          g2.setPaint(new GradientPaint(0, 0, Tema.FONDO_ARRIBA, 0, alto, Tema.FONDO_ABAJO));
79          g2.fillRect(0, 0, ancho, alto);
```

Líneas 72-79 El fondo: un degradado vertical de arriba abajo...

```
80          // Resplandor morado suave en el centro.
81          g2.setPaint(new RadialGradientPaint(new Point2D.Float(ancho / 2f, alto * 0.55f), ancho * 0.65f,
82                  new float[]{0f, 1f},
83                  new Color[]{Tema.conAlfa(Tema.MORADO, 40), Tema.conAlfa(Tema.MORADO, 0)}));
84          g2.fillRect(0, 0, ancho, alto);
85
```

Líneas 80-85 ...un resplandor radial morado en el centro (`RadialGradientPaint`)...

```
86          // Adornos: X y O grandes y desvaidas a los lados, y rejillas de puntos.
87          g2.setColor(Tema.DECORACION);
88          g2.setStroke(Dibujo.redondo(20f));
89          g2.draw(equis(ancho * 0.07, alto * 0.5, 100));
90          g2.draw(new Ellipse2D.Double(ancho * 0.94 - 60, alto * 0.36 - 60, 120, 120));
91          g2.draw(new Ellipse2D.Double(ancho * 0.07 - 56, alto * 0.82 - 56, 112, 112));
92          g2.draw(equis(ancho * 0.93, alto * 0.72, 90));
93          pintarPuntos(g2, ancho * 0.04, alto * 0.22);
94          pintarPuntos(g2, ancho * 0.88, alto * 0.88);
95          g2.dispose();
96      }
97
```

Líneas 86-97 ...y los adornos del diseño: X y O gigantes y desvaídas a los lados y dos rejillas de puntos. Las posiciones son proporciones del tamaño (por ejemplo, `ancho * 0.07`).

```
98      private static Path2D equis(double cx, double cy, double tam) {
99          double r = tam / 2;
100         Path2D equis = new Path2D.Double();
101         equis.moveTo(cx - r, cy - r);
102         equis.lineTo(cx + r, cy + r);
103         equis.moveTo(cx + r, cy - r);
104         equis.lineTo(cx - r, cy + r);
105         return equis;
106     }
107
```

Líneas 98-107 Ayudante: una X como ruta.

```
108     private static void pintarPuntos(Graphics2D g2, double x, double y) {
109         g2.setColor(Tema.conAlfa(Tema.LAVANDA, 45));
110         for (int fila = 0; fila < 3; fila++) {
111             for (int columna = 0; columna < 3; columna++) {
112                 g2.fill(new Ellipse2D.Double(x + columna * 18, y + fila * 18, 6, 6));
113             }
114         }
115     }
116 }
```

Líneas 108-116 Ayudante: una rejilla de 3×3 puntitos.

PantallaMenu.java nuevo

Pantalla del menú (diseño 1) · 34 líneas · `Totito/src/PantallaMenu.java`

Jugar, Instrucciones y Salir dentro de una tarjeta.

```
1  import java.awt.Dimension;
2  import javax.swing.BorderFactory;
3  import javax.swing.Box;
4  import javax.swing.BoxLayout;
5
```

Líneas 1-5 Imports para el layout de la tarjeta.

```
6  /**
7   * Pantalla 1 del diseno: menu principal con Jugar, Instrucciones y Salir.
8   */
9  public class PantallaMenu extends PantallaBase {
10
```

Líneas 6-10 Hereda de `PantallaBase`: el fondo y el encabezado ya vienen hechos.

```
11     public PantallaMenu(Navegacion navegacion) {
12         super(navegacion);
13
```

Líneas 11-13 Recibe la navegación y se la pasa a la base.

```
14         BotonNeon jugar = BotonNeon.menu("Jugar", Iconos.JUGAR, Tema.ROSA);
15         jugar.addActionListener(e -> navegacion.irAConfiguracion());
16         BotonNeon instrucciones = BotonNeon.menu("Instrucciones", Iconos.LIBRO, Tema.AZUL);
17         instrucciones.addActionListener(e -> navegacion.irAInstrucciones());
18         BotonNeon salir = BotonNeon.menu("Salir", Iconos.SALIR, Tema.MORADO);
19         salir.addActionListener(e -> navegacion.salir());
20
```

Líneas 14-20 Los tres botones del menú, cada uno con su color e icono. `addActionListener(e -> ...)` usa una lambda: "cuando lo pulsen, haz esto". Cada uno llama a un método de `Navegacion`.

```
21         Tarjeta tarjeta = new Tarjeta(Tema.MORADO);
22         tarjeta.setLayout(new BoxLayout(tarjeta, BoxLayout.Y_AXIS));
23         tarjeta.setBorder(BorderFactory.createEmptyBorder(34, 28, 34, 28));
24         tarjeta.add(jugar);
25         tarjeta.add(Box.createRigidArea(new Dimension(0, 14)));
26         tarjeta.add(instrucciones);
27         tarjeta.add(Box.createRigidArea(new Dimension(0, 14)));
28         tarjeta.add(salir);
29
```

Líneas 21-29 La tarjeta morada con `BoxLayout` vertical, un borde vacío como relleno interior y separaciones de 14 px entre botones.

```
30         relleno();
31         agregar(tarjeta);
32         relleno();
33     }
34 }
```

Líneas 30-34 Relleno arriba y abajo: la tarjeta queda centrada en vertical.

PantallaInstrucciones.java nuevo

Pantalla "Cómo jugar" (diseño 2) · 57 líneas · `Totito/src/PantallaInstrucciones.java`

Título con libro, tres pasos numerados, un tablero de ejemplo y el botón Regresar.

```
1  import java.awt.Dimension;
2  import javax.swing.BorderFactory;
3  import javax.swing.Box;
4  import javax.swing.BoxLayout;
5
```

Líneas 1-5 Imports.

```
6  /**
7   * Pantalla 2 del diseno: "Cómo jugar", tres pasos y un tablero de ejemplo.
8   */
9  public class PantallaInstrucciones extends PantallaBase {
10
```

Líneas 6-10 Declaración.

```
11     private static final int ANCHO_PASOS = 400;
12
```

Líneas 11-12 Ancho de la columna de texto.

```
13     public PantallaInstrucciones(Navegacion navegacion) {
14         super(navegacion);
15
```

Líneas 13-15 Constructor.

```
16         String x = "<font color='" + Tema.hex(Tema.ROSA) + "'><b>X</b></font>";
17         String o = "<font color='" + Tema.hex(Tema.AZUL) + "'><b>O</b></font>";
18
```

Líneas 16-18 Fragmentos HTML para la X rosa y la O azul. Se arman con `Tema.hex`, así que usan exactamente los colores del tema.

```
19         Tarjeta tarjeta = new Tarjeta(Tema.MORADO);
20         tarjeta.setLayout(new BoxLayout(tarjeta, BoxLayout.Y_AXIS));
21         tarjeta.setBorder(BorderFactory.createEmptyBorder(20, 30, 22, 30));
```

Líneas 19-21 La tarjeta morada que contiene todo.

```
22         tarjeta.add(Rotulo.titulo("Cómo jugar", 36f, Tema.TEXTO, Tema.LAVANDA, ANCHO_PASOS, 70)
23                 .conIcono(Iconos.LIBRO, Tema.ROSA)
24                 .conSubrayado(Tema.ROSA, Tema.AZUL));
25         tarjeta.add(Box.createRigidArea(new Dimension(0, 6)));
```

Líneas 22-25 Título "Cómo jugar" con el icono del libro y subrayado de rosa a azul, como en el diseño.

```
26          tarjeta.add(new PasoInstruccion(1, Tema.ROSA,
27                  "Dos jugadores se turnan<br>para marcar " + x + " y " + o + ".", ANCHO_PASOS));
28          tarjeta.add(new PasoInstruccion(2, Tema.MORADO,
29                  "Gana quien forme una<br>línea de 3 símbolos.", ANCHO_PASOS));
30          tarjeta.add(new PasoInstruccion(3, Tema.AZUL,
31                  "Si el tablero se llena<br>sin línea, hay empate.", ANCHO_PASOS));
32          tarjeta.add(Box.createRigidArea(new Dimension(0, 10)));
```

Líneas 26-32 Los tres pasos, cada uno con su color de círculo (rosa, morado, azul) y su texto.

```
33          tarjeta.add(tableroDeEjemplo());
34
```

Líneas 33-34 El tablero de ejemplo, más pequeño (casillas de 46 px).

```
35          BotonNeon regresar = BotonNeon.pildora("Regresar", Iconos.FLECHA_IZQUIERDA,
36                  Tema.ROSA, Tema.MORADO, 330, 70);
37          regresar.addActionListener(e -> navegacion.irAlMenu());
38
```

Líneas 35-38 Botón píldora que vuelve al menú.

```
39          relleno();
40          agregar(tarjeta);
41          espacio(8);
42          agregar(regresar);
43          relleno();
44      }
45
```

Líneas 39-45 Se apilan la tarjeta y el botón, centrados en vertical.

```
46      /** El mismo dibujo del diseno: O X O en la fila de arriba y una X al centro. */
47      private static PanelTablero tableroDeEjemplo() {
48          Tablero ejemplo = new Tablero();
49          ejemplo.marcarCasilla(0, 0, 'O');
50          ejemplo.marcarCasilla(0, 1, 'X');
51          ejemplo.marcarCasilla(0, 2, 'O');
52          ejemplo.marcarCasilla(1, 1, 'X');
53          PanelTablero panel = new PanelTablero(46);
54          panel.mostrar(ejemplo);
55          return panel;
56      }
57  }
```

Líneas 46-57 Crea un `Tablero` real (la clase original), marca la posición del diseño (O X O arriba y X al centro) y se lo da a un `PanelTablero` que no acepta clics.

PantallaConfiguracion.java nuevo

Pantalla de configuración (diseño 3) · 69 líneas · `Totito/src/PantallaConfiguracion.java`

Nombres, símbolo y modo de juego antes de empezar.

```
1  /**
2   * Pantalla 3 del diseno: nombres de los jugadores, simbolo de cada uno y
3   * modo de juego, antes de empezar.
4   *
5   * El diseno trae un solo modo ("2 jugadores"); se agrego "Contra la IA"
6   * para no perder la IA_Minimax que ya tenia el proyecto.
7   */
8  public class PantallaConfiguracion extends PantallaBase {
9
```

Líneas 1-9 El comentario explica por qué se agregó "Contra la IA": el diseño solo traía "2 jugadores" y así no se perdía la IA del proyecto.

```
10     private static final int ANCHO_SECCION = 560;
11
```

Líneas 10-11 Ancho de los títulos de sección.

```
12     private final Navegacion navegacion;
13     private final TarjetaJugador jugador1;
14     private final TarjetaJugador jugador2;
15     private final SelectorSimbolo selector;
16     private final OpcionModo opcionDosJugadores;
17     private final OpcionModo opcionContraIA;
18     private Partida.Modo modo;
19
```

Líneas 12-19 Referencias a los componentes que hay que leer al final y el modo elegido.

```
20     public PantallaConfiguracion(Navegacion navegacion) {
21         super(navegacion);
22         this.navegacion = navegacion;
23
```

Líneas 20-23 Constructor.

```
24         jugador1 = new TarjetaJugador("Jugador 1", Tema.FRESA);
25         jugador2 = new TarjetaJugador("Jugador 2", Tema.AZUL);
26         selector = new SelectorSimbolo();
```

Líneas 24-26 Las dos tarjetas de jugador (fresa y azul) y el selector de símbolo.

```
27         opcionDosJugadores = new OpcionModo(Iconos.PERSONAS, "2 jugadores", "Turnos en el mismo equipo");
28         opcionContraIA = new OpcionModo(Iconos.COMPUTADORA, "Contra la IA", "La computadora usa Minimax");
29         opcionDosJugadores.setAlSeleccionar(() -> elegirModo(Partida.Modo.DOS_JUGADORES));
30         opcionContraIA.setAlSeleccionar(() -> elegirModo(Partida.Modo.CONTRA_IA));
31
```

Líneas 27-31 Las dos opciones de modo. Cada una, al pulsarse, llama a `elegirModo` con su valor.

```
32          BotonNeon comenzar = BotonNeon.pildora("Comenzar partida", Iconos.JUGAR, Tema.ROSA, Tema.MORADO, 440, 6
    6);
33          comenzar.addActionListener(e -> comenzar());
34          BotonNeon volver = BotonNeon.contorno("Volver", Iconos.FLECHA_IZQUIERDA, 300, 52);
35          volver.addActionListener(e -> navegacion.irAlMenu());
36
```

Líneas 32-36 "Comenzar partida" (píldora rosa→morado) y "Volver" (contorno).

```
37          relleno();
38          agregar(Rotulo.seccion("JUGADORES", null, ANCHO_SECCION));
39          agregar(fila(8, jugador1, jugador2));
40          espacio(2);
41          agregar(Rotulo.seccion("SÍMBOLOS", "Cada jugador elige su símbolo", ANCHO_SECCION));
42          agregar(selector);
43          espacio(4);
44          agregar(Rotulo.seccion("MODO DE JUEGO", null, ANCHO_SECCION));
45          agregar(fila(6, opcionDosJugadores, opcionContraIA));
46          espacio(4);
47          agregar(comenzar);
48          agregar(volver);
49          relleno();
50
```

Líneas 37-50 El orden en pantalla, de arriba abajo: JUGADORES, las dos tarjetas en fila, SÍMBOLOS, el selector, MODO DE JUEGO, las dos opciones en fila y los dos botones.

```
51          elegirModo(Partida.Modo.DOS_JUGADORES);
52      }
53
```

Líneas 51-53 Al abrir, queda elegido "2 jugadores".

```
54      private void elegirModo(Partida.Modo nuevoModo) {
55          modo = nuevoModo;
56          opcionDosJugadores.setSeleccionada(nuevoModo == Partida.Modo.DOS_JUGADORES);
57          opcionContraIA.setSeleccionada(nuevoModo == Partida.Modo.CONTRA_IA);
58          jugador2.setComputadora(nuevoModo == Partida.Modo.CONTRA_IA);
59      }
60
```

Líneas 54-60 Marca una opción y desmarca la otra. Si es contra la IA, convierte la tarjeta del Jugador 2 en "Computadora".

```
61      private void comenzar() {
62          char fichaJugador1 = selector.getFichaJugador1();
63          char fichaJugador2 = fichaJugador1 == 'X' ? 'O' : 'X';
64          navegacion.iniciarPartida(
65                  new Jugador(jugador1.getNombre(), fichaJugador1),
66                  new Jugador(jugador2.getNombre(), fichaJugador2),
67                  modo);
68      }
69  }
```

Líneas 61-69 El Jugador 2 recibe la ficha contraria a la del 1. Se crean los dos `Jugador` (clase original) y se pide a la navegación que empiece la partida.

PantallaJuego.java nuevo

Pantalla de la partida · 134 líneas · `Totito/src/PantallaJuego.java`

Marcador, turno y tablero. Hace el papel de controlador que antes tenía `JuegoMain`.

```
1  import java.awt.Color;
2  import javax.swing.Timer;
3
```

Líneas 1-3 Imports: `javax.swing.Timer` se usa para las pausas.

```
4  /**
5   * Pantalla de la partida: marcador, de quien es el turno y el tablero.
6   * (El diseno no trae esta pantalla; se hizo con el mismo estilo.)
7   *
8   * Hace el papel que tenia JuegoMain en la version original: recibe los
9   * clics, se los pasa a la Partida y, en el modo contra la IA, espera un
10  * momento y hace jugar a la computadora. Cuando la partida termina deja
11  * ver la raya ganadora un instante y pasa a la pantalla de resultado.
12  */
13 public class PantallaJuego extends PantallaBase {
14
```

Líneas 4-14 El comentario explica la relación con `JuegoMain`.

```
15     /** Pausa antes de que juegue la IA, para que el cambio de turno se note. */
16     private static final int PAUSA_IA_MS = 450;
17     /** Tiempo que se ve la raya ganadora antes de cambiar de pantalla. */
18     private static final int PAUSA_RESULTADO_MS = 1200;
19
```

Líneas 15-19 Las dos pausas: 450 ms antes de que juegue la IA (la original usaba 350 ms), para que el cambio de turno se note, y 1200 ms mostrando la raya ganadora antes de pasar al resultado.

```
20     private final Navegacion navegacion;
21     private final PanelMarcador marcador = new PanelMarcador();
22     private final Rotulo turno = Rotulo.titulo("", 26f, Tema.TEXTO, Tema.TEXTO, 590, 44);
23     private final PanelTablero tablero = new PanelTablero(100);
24
```

Líneas 20-24 Los tres componentes de la pantalla: marcador, letrero de turno y tablero (casillas de 100 px).

```
25     private Partida partida;
26     /** Temporizador en espera (jugada de la IA o paso al resultado), o null. */
27     private Timer pendiente;
28
```

Líneas 25-28 La partida que se está mostrando y el temporizador pendiente (si hay uno).

```
29     public PantallaJuego(Navegacion navegacion) {
30         super(navegacion);
31         this.navegacion = navegacion;
32         tablero.setOyente(this::alPulsarCasilla);
33
```

Líneas 29-33 Conecta el tablero con esta pantalla: `this::alPulsarCasilla` cumple la interfaz `OyenteCasilla`.

```
34          BotonNeon reiniciar = BotonNeon.contorno("Reiniciar", Iconos.REINICIAR, 250, 66);
35          reiniciar.addActionListener(e -> {
36              partida.reiniciar();
37              comenzar(partida);
38          });
39          BotonNeon menu = BotonNeon.pildora("Menú principal", Iconos.CASA, Tema.AZUL, Tema.MORADO, 270, 66);
40          menu.addActionListener(e -> {
41              cancelarPendiente();
42              navegacion.irAlMenu();
43          });
44
```

Líneas 34-44 "Reiniciar" limpia el tablero de esta misma partida (el marcador se queda). "Menú principal" cancela cualquier pausa pendiente y vuelve al menú.

```
45          relleno();
46          agregar(marcador);
47          espacio(2);
48          agregar(turno);
49          agregar(tablero);
50          espacio(6);
51          agregar(fila(8, reiniciar, menu));
52          relleno();
53      }
54
```

Líneas 45-54 Orden en pantalla: marcador, turno, tablero y la fila de botones.

```
55      /** Muestra la partida (nueva o reiniciada) y, si empieza la IA, la pone a jugar. */
56      public void comenzar(Partida partida) {
57          cancelarPendiente();
58          this.partida = partida;
59          actualizar();
60          programarTurnoIA();
61      }
62
```

Líneas 55-62 Punto de entrada desde `PanelPrincipal`: cancela pausas viejas, guarda la partida, refresca la pantalla y, si abre la IA, la pone a jugar.

```
63      private void alPulsarCasilla(int fila, int columna) {
64          if (pendiente != null || partida.esTurnoDeIA()) {
65              return;   // mientras la IA "piensa" no se aceptan clics
66          }
67          if (partida.jugar(fila, columna)) {
68              despuesDeJugada();
69          }
70      }
71
```

Líneas 63-71 **Qué pasa con un clic.** Si hay una pausa en curso o es turno de la IA, se ignora. Si no, se intenta la jugada; si valió, sigue `despuesDeJugada`.

```
72      private void despuesDeJugada() {
73          actualizar();
74          if (partida.getEstado() == Partida.Estado.EN_CURSO) {
75              programarTurnoIA();
76          } else {
77              esperar(PAUSA_RESULTADO_MS, () -> navegacion.mostrarResultado(partida));
78          }
79      }
80
```

Líneas 72-80 Refresca la pantalla. Si la partida sigue, quizá le toque a la IA; si terminó, espera 1.2 s (para que se vea la raya) y pasa al resultado.

```
81      private void programarTurnoIA() {
82          if (partida.esTurnoDeIA()) {
83              esperar(PAUSA_IA_MS, () -> {
84                  partida.jugarTurnoIA();
85                  despuesDeJugada();
86              });
87          }
88      }
89
```

Líneas 81-89 Si le toca a la IA, espera la pausa, le pide la jugada a `Partida` y continúa igual que tras un clic.

```
90      /** Ejecuta la accion una sola vez, dentro de "ms" milisegundos. */
91      private void esperar(int ms, Runnable accion) {
92          cancelarPendiente();
93          pendiente = new Timer(ms, e -> {
94              pendiente = null;
95              accion.run();
96          });
97          pendiente.setRepeats(false);
98          pendiente.start();
99      }
100
```

Líneas 90-100 Crea un `javax.swing.Timer` de una sola vez (`setRepeats(false)`). Se usa este Timer y no `Thread.sleep` porque dormir congelaría la ventana; este Timer ejecuta la acción en el hilo de Swing sin bloquear nada.

```
101     /** Evita que un temporizador viejo actue sobre una partida que ya se reinicio o se dejo. */
102     private void cancelarPendiente() {
103         if (pendiente != null) {
104             pendiente.stop();
105             pendiente = null;
106         }
107     }
108
```

Líneas 101-108 Detiene el temporizador pendiente. Evita un error sutil: que, después de pulsar Reiniciar o Menú, una jugada vieja de la IA caiga sobre una partida nueva.

```
109     /** Refresca tablero, marcador y el letrero del turno segun la partida. */
110     private void actualizar() {
111         tablero.mostrar(partida.getTablero());
112         marcador.mostrar(partida);
113 
114         Jugador enTurno = partida.getJugadorEnTurno();
115         boolean enCurso = partida.getEstado() == Partida.Estado.EN_CURSO;
116         tablero.setInteractivo(enCurso && !partida.esTurnoDeIA());
117         tablero.setFichaFantasma(enTurno.getFicha());
118 
```

Líneas 109-118 Pasa el estado de la partida a la pantalla: tablero, marcador, si el tablero acepta clics (no durante el turno de la IA) y qué ficha mostrar como fantasma.

```
119         switch (partida.getEstado()) {
120             case EN_CURSO -> {
121                 Color color = Tema.colorDeFicha(enTurno.getFicha());
122                 String texto = partida.esTurnoDeIA()
123                         ? "La computadora está pensando…"
124                         : "Turno de " + enTurno.getNombre();
125                 turno.setTexto(texto, Tema.aclarar(color, 0.35f), color);
126             }
127             case VICTORIA -> {
128                 Color color = Tema.colorDeFicha(partida.getGanador().getFicha());
129                 turno.setTexto("¡Tres en línea!", Tema.aclarar(color, 0.35f), color);
130             }
131             case EMPATE -> turno.setTexto("¡Empate!", Tema.LAVANDA, Tema.MORADO);
132         }
133     }
134 }
```

Líneas 119-134 El letrero: "Turno de …" o "La computadora está pensando…" mientras la partida sigue; "¡Tres en línea!" o "¡Empate!" al terminar. Siempre en el color de la ficha correspondiente.

PantallaResultado.java nuevo

Pantalla de resultado (diseño 4) · 125 líneas · `Totito/src/PantallaResultado.java`

"¡Ganó ...!" con trofeo, confeti y el tablero final con la raya. También muestra el empate.

```
1  import java.awt.Color;
2  import java.awt.Dimension;
3  import java.awt.Graphics2D;
4  import java.awt.geom.AffineTransform;
5  import java.awt.geom.RoundRectangle2D;
6  import java.util.Random;
7  import javax.swing.BorderFactory;
8  import javax.swing.Box;
9  import javax.swing.BoxLayout;
10
```

Líneas 1-10 Imports.

```
11 /**
12  * Pantalla 4 del diseno: "¡Ganó Jugador 2!" con trofeo, confeti y el
13  * tablero final con la raya ganadora. Tambien sirve para el empate
14  * (sin trofeo ni confeti).
15  */
16 public class PantallaResultado extends PantallaBase {
17
```

Líneas 11-17 El comentario lo describe.

```
18     private static final int ANCHO_INTERIOR = 440;
19
```

Líneas 18-19 Ancho del contenido de la tarjeta.

```
20     private final IconoResultado icono = new IconoResultado();
21     private final Rotulo titulo = Rotulo.titulo("", 40f, Tema.CIAN, Tema.AZUL, ANCHO_INTERIOR, 54);
22     private final Rotulo frase = Rotulo.leyenda("", ANCHO_INTERIOR);
23     private final PanelTablero tablero = new PanelTablero(66);
24     private final TarjetaConConfeti tarjeta = new TarjetaConConfeti();
25
```

Líneas 20-25 Los componentes que cambian en cada resultado: ilustración, título, frase, tablero (casillas de 66 px) y la tarjeta con confeti.

```
26     public PantallaResultado(Navegacion navegacion) {
27         super(navegacion);
28
```

Líneas 26-28 Constructor.

```
29         tarjeta.setLayout(new BoxLayout(tarjeta, BoxLayout.Y_AXIS));
30         tarjeta.setBorder(BorderFactory.createEmptyBorder(14, 20, 18, 20));
31         tarjeta.add(icono);
32         tarjeta.add(titulo);
33         tarjeta.add(frase);
34         tarjeta.add(Box.createRigidArea(new Dimension(0, 6)));
35         tarjeta.add(tablero);
36
```

Líneas 29-36 La tarjeta apila ilustración, título, frase y tablero.

```
37          BotonNeon jugarDeNuevo = BotonNeon.pildora("Jugar de nuevo", Iconos.REINICIAR,
38                  Tema.ROSA, Tema.MORADO, 290, 72);
39          jugarDeNuevo.addActionListener(e -> navegacion.jugarDeNuevo());
40          BotonNeon menu = BotonNeon.pildora("Menú principal", Iconos.CASA,
41                  Tema.AZUL, new Color(0x3A, 0x3F, 0xC8), 290, 72);
42          menu.addActionListener(e -> navegacion.irAlMenu());
43
```

Líneas 37-43 "Jugar de nuevo" (rosa→morado) y "Menú principal" (azul), como en el diseño.

```
44          relleno();
45          agregar(tarjeta);
46          espacio(6);
47          agregar(fila(6, jugarDeNuevo, menu));
48          relleno();
49      }
50
```

Líneas 44-50 Tarjeta arriba y la fila de botones debajo, centradas.

```
51      /** Prepara la pantalla con el resultado de la partida que acaba de terminar. */
52      public void mostrar(Partida partida) {
53          tablero.mostrar(partida.getTablero());
54          boolean empate = partida.getEstado() == Partida.Estado.EMPATE;
55          icono.setEmpate(empate);
56          tarjeta.setConConfeti(!empate);
57
```

Líneas 51-57 Se llama cada vez que termina una partida. Copia el tablero final (con su raya) y elige trofeo o empate.

```
58          if (empate) {
59              titulo.setTexto("¡Empate!", Tema.LAVANDA, Tema.MORADO);
60              frase.setTexto("Nadie cedió ni una casilla");
61              return;
62          }
```

Líneas 58-62 Empate: título lavanda y frase propia, sin confeti.

```
63          Jugador ganador = partida.getGanador();
64          Color color = Tema.colorDeFicha(ganador.getFicha());
65          titulo.setTexto("¡Ganó " + ganador.getNombre() + "!", Tema.aclarar(color, 0.35f), color);
66          if (partida.esComputadora(ganador)) {
67              frase.setTexto("La máquina no perdona");
68          } else if (partida.getModo() == Partida.Modo.CONTRA_IA) {
69              frase.setTexto("¡Le ganaste a la máquina!");
70          } else {
71              frase.setTexto("Excelente jugada");
72          }
73      }
74
```

Líneas 63-74 Victoria: el título lleva el nombre del ganador en el color de su ficha. La frase depende de quién ganó: la computadora, un humano contra la IA o un humano contra otro humano.

```java
    /**
     * Tarjeta con papelitos de colores. Las posiciones salen de un Random
     * con semilla fija: siempre son las mismas y no "bailan" al repintar.
     */
    private static class TarjetaConConfeti extends Tarjeta {

        private static final Color[] COLORES = {Tema.ROSA, Tema.CIAN, Tema.MORADO};
        private boolean conConfeti = true;

        TarjetaConConfeti() {
            super(Tema.MORADO);
        }

        void setConConfeti(boolean conConfeti) {
            this.conConfeti = conConfeti;
            repaint();
        }

```

Líneas 75-92 Clase interna: una `Tarjeta` que dibuja confeti. Solo la usa esta pantalla, por eso va aquí dentro.

```java
        @Override
        protected void pintarDecoracion(Graphics2D g2, RoundRectangle2D forma) {
            if (!conConfeti) {
                return;
            }
            Random azar = new Random(11);
            double centro = forma.getCenterX();
            int colocados = 0;
```

Líneas 93-100 Sobrescribe el gancho de `Tarjeta`. Usa un `Random` con **semilla fija** (11), así que los papelitos salen siempre en el mismo lugar y no "bailan" cada vez que se redibuja.

```java
            while (colocados < 22) {
                double x = forma.getX() + 18 + azar.nextDouble() * (forma.getWidth() - 36);
                double y = forma.getY() + 16 + azar.nextDouble() * (forma.getHeight() - 32);
                // Solo arriba (alrededor del trofeo) o a los lados del tablero.
                boolean libre = y < forma.getY() + 100 || Math.abs(x - centro) > 135;
                boolean sobreTrofeo = y < forma.getY() + 120 && Math.abs(x - centro) < 60;
                double angulo = azar.nextDouble() * Math.PI;
                Color color = COLORES[azar.nextInt(COLORES.length)];
                if (!libre || sobreTrofeo) {
                    continue;
                }
```

Líneas 101-111 Genera posiciones al azar y solo acepta las que caen arriba o a los lados del tablero, y nunca encima del trofeo. Hasta tener 22 papelitos.

```
112                 AffineTransform previo = g2.getTransform();
113                 g2.translate(x, y);
114                 g2.rotate(angulo);
115                 RoundRectangle2D papel = new RoundRectangle2D.Double(-3.5, -7, 7, 14, 3, 3);
116                 g2.setColor(Tema.conAlfa(color, 45));
117                 g2.fill(new RoundRectangle2D.Double(-6, -9.5, 12, 19, 6, 6));
118                 g2.setColor(Tema.conAlfa(color, 220));
119                 g2.fill(papel);
120                 g2.setTransform(previo);
121                 colocados++;
122             }
123         }
124     }
125 }
```

Líneas 112–125 Para dibujar un papelito girado se traslada y rota el sistema de coordenadas (`translate` y `rotate`), se dibuja un rectángulo con un halo suave y se restaura la transformación anterior.

Parte G — Configuración del proyecto

Archivos que no son código Java pero hacen que el proyecto abra, compile y se entienda.

pom.xml nuevo

Proyecto Maven · 39 líneas · `Totito/pom.xml`

Le dice a NetBeans y a Maven cómo compilar y empaquetar el juego. Sigue la forma de los demás proyectos del repositorio.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:sch
   emaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
3      <modelVersion>4.0.0</modelVersion>
4      <groupId>com.mycompany</groupId>
5      <artifactId>Totito</artifactId>
6      <version>1.0-SNAPSHOT</version>
7      <packaging>jar</packaging>
```

Líneas 1-7 Encabezado XML estándar de Maven y la identidad del proyecto: `groupId` `com.mycompany` (igual que el resto del repositorio), `artifactId` Totito, versión y empaquetado `jar`.

```
8      <properties>
9          <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
10         <!-- Funciona con JDK 17 o mas nuevo. -->
11         <maven.compiler.release>17</maven.compiler.release>
12         <!-- Clase que arranca el juego (boton Run de NetBeans). -->
13         <exec.mainClass>TotitoApp</exec.mainClass>
14     </properties>
```

Líneas 8-14 Codificación UTF-8 (necesaria por las tildes), compilar para **Java 17** (funciona con cualquier JDK 17 o más nuevo) y `exec.mainClass`: la clase que ejecuta el botón Run de NetBeans.

```
15     <build>
16         <!-- El codigo vive en src/ (sin paquetes), no en src/main/java. -->
17         <sourceDirectory>src</sourceDirectory>
```

Líneas 15-17 `sourceDirectory` = `src`: el código quedó en `src/` como venía en el zip, en lugar de la carpeta habitual de Maven (`src/main/java`). Así no hubo que mover el código original.

```
18         <plugins>
19             <plugin>
20                 <groupId>org.apache.maven.plugins</groupId>
21                 <artifactId>maven-compiler-plugin</artifactId>
22                 <version>3.13.0</version>
23             </plugin>
24             <plugin>
```

Líneas 18-24 Versión fija del compilador (3.13.0): las versiones viejas que trae Maven por defecto no entienden `maven.compiler.release`.

```
                <!-- Deja el .jar ejecutable con doble clic o con java -jar. -->
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.4.2</version>
                <configuration>
                    <archive>
                        <manifest>
                            <mainClass>TotitoApp</mainClass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Líneas 25–39 El plugin de jar escribe `Main-Class: TotitoApp` en el manifiesto. Por eso el .jar se abre con doble clic o con `java -jar`.

Glosario de Java y Swing

Término	Qué significa aquí
`JFrame`	La ventana del sistema operativo. Solo hay una: `TotitoApp`.
`JPanel` / `JComponent`	Piezas rectangulares dentro de la ventana. Un JPanel puede contener otras piezas; casi todos los componentes del juego heredan de uno de los dos.
`paintComponent(Graphics g)`	Método que Swing llama cuando hay que dibujar un componente. Al sobrescribirlo, se decide cómo se ve.
`repaint()`	Avisa a Swing de que un componente cambió. Swing llamará a `paintComponent` en cuanto pueda (no al instante).
`Graphics2D`	El «lápiz» de Java2D: dibuja formas (`draw`), las rellena (`fill`) y escribe texto (`drawString`).
`Shape`, `Path2D`, `Ellipse2D`, `RoundRectangle2D`, `Arc2D`, `Area`	Figuras geométricas. `Path2D` se arma punto por punto (`moveTo`, `lineTo`, `quadTo`). `Area` permite sumar, restar e intersectar figuras.
`BasicStroke`	El tipo de trazo: grosor y forma de puntas y esquinas.
`GradientPaint` / `RadialGradientPaint`	Rellenos degradados, lineales o circulares.
Alfa (transparencia)	Cuarto número de un `Color`: 0 es invisible y 255 opaco. Es la base del cristal y del neón.
`AlphaComposite`	Hace que todo lo que se dibuje después salga semitransparente.
Antialiasing	Suavizado de bordes, para que las curvas no se vean «en escalera». Se activa en `Dibujo.preparar`.
`BorderLayout`, `BoxLayout`, `CardLayout`	Organizadores de posición: por zonas (norte, centro…), en fila o columna, y uno a la vez como un mazo de cartas.
Layout `null` + `doLayout()`	Posicionar hijos a mano con `setBounds`. Se usa donde hay un solo hijo en un lugar exacto.
Glue / RigidArea	Espacios dentro de un BoxLayout: el glue es elástico (centra), el rigid area es fijo.
`javax.swing.Timer`	Ejecuta una acción después de un tiempo, en el hilo de Swing y sin congelar la ventana.
Event Dispatch Thread	El único hilo desde el que se debe tocar la interfaz. `SwingUtilities.invokeLater` manda trabajo a ese hilo.
`MouseAdapter`	Clase para escuchar el ratón (`mousePressed`, `mouseMoved`…) sobrescribiendo solo lo necesario.
`interface`	Contrato de métodos sin implementación. Aquí: `Navegacion`, `OyenteCasilla`, `Icono`.
Lambda `(a, b) -> {...}`	Una función escrita en línea. Se usa para las acciones de los botones y para cada icono.
Referencia a método `this::cerrar`	Atajo para pasar un método existente como si fuera una lambda.

`enum`	Tipo con valores fijos: `Partida.Modo`, `Partida.Estado`, `BotonNeon.Estilo`.
Método de fábrica	Método estático que crea objetos ya configurados (`BotonNeon.menu(...)`), en lugar de llamar directamente al constructor.
`static` / `final`	`static`: pertenece a la clase y no a cada objeto. `final`: no se puede cambiar (una constante) ni heredar (una clase).
Minimax	Algoritmo de juegos por turnos: prueba todas las jugadas futuras suponiendo que cada lado juega perfecto; maximiza en su turno y minimiza en el del rival.

Pruebas realizadas

Antes de entregar se verificó lo siguiente, con programas de prueba que no forman parte del proyecto:

Prueba	Resultado
Compilar con `javac` para Java 17	Sin errores (probado con el JDK 21).
Compilar y empaquetar con `mvn package`	Genera `Totito-1.0-SNAPSHOT.jar` con `Main-Class: TotitoApp`.
Código original	Comparado con Git: `Tablero`, `Jugador`, `IA_Minimax` y `JuegoMain` no tienen ningún cambio.
Reglas de `Partida`	Empieza la X aunque la tenga el Jugador 2; se rechazan casillas ocupadas; los turnos alternan; victoria con su línea; no se juega con la partida terminada; marcador; reinicio que conserva el marcador; empate; dos jugadores con la misma ficha.
La IA nunca pierde	Se recorrieron por programa **las 642 partidas completas posibles** contra la IA, con el humano empezando y con la IA empezando. En ninguna gana el humano.
Flujo real de la interfaz	Modo IA con la IA abriendo: la computadora juega sola tras la pausa; clics simulados sobre el tablero ponen la ficha del humano; la partida termina y pasa a la pantalla de resultado.
Aspecto	Las 6 pantallas se dibujaron a imagen (son las capturas de este documento) y se compararon con el diseño. Se corrigieron tres detalles: la etiqueta «Nombre» encimada, los destellos del trofeo cortados y unas líneas que tocaban el borde.

Total: 658 comprobaciones, 0 fallos. Lo único que no se pudo hacer desde el servidor es abrir la ventana en vivo, porque el servidor no tiene pantalla; las capturas salen del mismo código dibujado sin ventana.

Cómo seguir trabajando

Quiero…	Dónde y cómo
Cambiar colores o la letra	`Tema.java`. Todo sale de ahí.
Agregar una pantalla	Crear `class PantallaX extends PantallaBase`; en el constructor usar `agregar`, `espacio` y `relleno`. Registrarla en `PanelPrincipal` con `add(new PantallaX(this), "x")` y agregar el método para mostrarla en `Navegacion`.
Agregar un botón	`BotonNeon.pildora("Texto", Iconos.ALGO, color1, color2, ancho, alto)` y `addActionListener(e -> ...)`.
Agregar un icono	Copiar una constante de `Iconos.java`; las medidas van como fracción de `t`.
Cambiar reglas o quién empieza	`Partida.java`: `FICHA_INICIAL`, `jugar()`, `reiniciar()`.
Cambiar las pausas	`PantallaJuego.PAUSA_IA_MS` y `PAUSA_RESULTADO_MS`.
Cambiar el tamaño de la ventana	`PanelPrincipal.ANCHO` / `ALTO`; después revisar que todo quepa, sobre todo la configuración, que es la pantalla más llena.

Ideas pendientes

- Niveles de dificultad: que la IA a veces juegue al azar en lugar de usar Minimax (en `Partida.jugarTurnoIA`).
- Alternar quién empieza en cada «Jugar de nuevo».
- Animar la raya ganadora y el confeti con un `javax.swing.Timer`.
- Sonidos al marcar y al ganar (`javax.sound.sampled`).
- Jugar con el teclado numérico (teclas 1–9).
- Guardar el marcador entre sesiones (por ejemplo, con `java.util.prefs.Preferences`).